

Silent Threshold Encryption with One-Shot Adaptive Security

Mathias Hall-Andersen^{1*}, Mark Simkin^{2**}, and Benedikt Wagner^{3***}

¹ ZKSecurity

² Flashbots

³ Ethereum Foundation

Abstract. Threshold encryption enables a sender to encrypt a message towards n recipients, such that any sufficiently large subset can decrypt the message, whereas any subset of too small size cannot. Silent threshold encryption additionally requires that all recipients can generate their public keys independently of each other, without engaging in an interactive distributed key generation protocol.

In this work, we present a simple blueprint for constructing such silent threshold encryption schemes, which remain secure as long as the number of corruptions is at most $t \leq (1/2 - \epsilon) \cdot n$, where $\epsilon > 0$ is an arbitrary constant. Our construction allows for ciphertexts and recipient public keys, whose sizes are independent of n . We evaluate the concrete efficiency of our construction and show that it is highly efficient. As an exemplary data point, when $t < n/3$, encrypting 1 MB results in a ciphertext of size 1.072 MB. On the technical side, we introduce a new model of corruptions, which we call *one-shot adaptive corruptions*, which conceptually lie between static and fully adaptive corruptions. We believe that the notion itself and our associated proof techniques may be of independent interest.

In comparison to prior works, we have smaller recipient public keys, do not require strong assumptions, such as indistinguishability obfuscation, or idealizing models, such as the generic group model, we allow for instantiating our blueprint to obtain plausible post-quantum security, and we prove security under a stronger security notion.

1 Introduction

Threshold encryption [Des88; DF90] allows a sender to encrypt a message towards a set of n recipients, such that any sufficiently large subset of recipients can jointly decrypt it, while any subset of too small size learns nothing about the message. The main advantage of such encryption schemes over their standard counterparts is their ability to distribute trust: the encrypted message remains hidden, even if some recipients are compromised or act maliciously.

Most existing threshold encryption [SG98; CGJ+99; JL00; BBH06] schemes require the recipients' keys to be correlated and thus jointly generated. The original works of Desmedt and Frankel [Des88; DF90] assumes a trusted dealer who distributes the necessary key material to all recipients. Later works by Pedersen [Ped91] and Gennaro et al. [GJK+99] demonstrate that this dealer can be replaced by an interactive protocol among the recipients, eliminating the need for a single trusted party during setup.

More recently, several works [RSY21; GKP+24] have investigated the possibility constructing *silent* threshold encryption schemes: ones that enable the recipients to generate their keys *independently*, while still allowing for the ciphertext size to be sub-linear in n . Reyzin, Smith, and Yakoubov [RSY21] show how to construct such a scheme with constant ciphertext size from indistinguishability obfuscation [BGI+01], albeit requiring a sender to possess a public key that is $\text{poly}(n)$ bits large. While their work is theoretically interesting, the size of the sender's public key and the use of obfuscation renders their solution impractical. Garg et al. [GKP+24] present a significantly more efficient construction with constant ciphertext size using pairings. Unfortunately, their construction suffers from several drawbacks. The authors only prove security in the generic group model [Sho97; Mau05]. Their construction is not secure against quantum adversaries [Sho94] due to their use of discrete logarithm based assumptions. Finally, the size of each recipient's public key is linear in the maximum number of potential recipients.⁴

* mathias@zksecurity.xyz

** mark@univariate.org

*** benedikt.wagner@ethereum.org

⁴ The authors show how all individual public keys can be jointly compressed into a short encryption key, but this would require interaction between the recipients during key generation.

Scheme	Sizes		Corruptions		Security	Assumptions	Post-Quantum
	ctxt	ek	Threshold	Type			
Naive	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$t < n/2$	Adaptive	IND-CCA2	PKE	✓
[GKP+24]	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$t < n/2$	Static	IND-CPA	GGM + Pairings	✗
[RSY21]	$\mathcal{O}(1)$	$\mathcal{O}(1)^6$	$t < n/2$	Static	IND-CPA	iO + OWFs	✗
Our work	$\mathcal{O}(\kappa)$	$\mathcal{O}(1)$	$t \leq (1/2 - \epsilon) \cdot n$	One-Shot Adaptive	IND-CCA1	IK-PKE + NIKE	✓

Table 1. Comparison of existing approaches towards silent threshold encryption in the random oracle model without an interactive distributed key generation phase. IND-CPA does not provide the adversary with a decryption oracle, IND-CCA1 provides it before seeing the challenge ciphertext and IND-CCA2 allows for decryption queries at all times. IK-PKE stands for public key encryption with key indistinguishability, formalized in Definition 9. NIKE stands for non-interactive key exchange, formalized in Definition 10. Ciphertext sizes are multiples of the security parameter. The naive construction encrypts a secret shares of the message towards all participants.

1.1 Our Contribution

In this work, we present a simple, but powerful blueprint for constructing silent threshold encryption. The recipients’ public keys in our construction are constant⁵ sized and can be generated independently of each other. Our ciphertext size is independent of the total number of recipients n . Decryption succeeds as long as sufficiently many honest decryption shares are provided, even if the adversary provides malformed shares on behalf of the corrupted parties.

We introduce a new notion called *one-shot adaptive security* and prove our construction secure under this new notion without resorting to generic group models. Broadly speaking, security notions for threshold encryption schemes differ along two axes: when the adversary can request partial decryptions of adversarial ciphertexts and when they can corrupt honest parties. In static security notions, corruptions occur before seeing the challenge ciphertext, whereas in adaptive security, they can occur after. Our notion of one-shot adaptive security allows arbitrary corruptions before seeing the challenge ciphertext and afterwards the adversary can *once* more provide a set of parties that it wants to corrupt. Both the notion itself as well as our associated proof techniques may be of independent interest.

Our construction can be instantiated to offer plausible post-quantum security. We evaluate its concrete efficiency under both classical and quantum-resistant instantiations, showing its practicality. As an exemplary data point, assuming at most $n/3$ of the recipients are corrupt, our encryption scheme allows for encrypting a 1 MB plaintext into a 1.431 MB or 1.072 MB ciphertext, depending on whether security against quantum adversaries is desired or not.

The main drawback of our construction lies in its restriction on the number of tolerated corrupt recipients. Let n be the total number of recipients and t be an upper bound on the number of corrupt recipients. If honest recipients should always be able to decrypt the message, then $t < n/2$ is a necessary condition for any scheme. In comparison to this, we require the slightly stronger constraint $t \leq (1/2 - \epsilon) \cdot n$, where ϵ is an arbitrarily small constant. A comparison of existing works and ours can be found in Table 1.

1.2 Application: Fair Exchange Without a Trusted Intermediary

To highlight the practicality of our construction, let us review a recent application of silent threshold encryption in the context of blockchains. The precise inner workings of blockchains are not important for the current exposition, suffice it to say that we have a block builder and a proposer, who would like to perform a fair exchange of digital goods. That is, each of the two parties holds some private data and they would both like to ensure that either both learn each other’s data or neither of them does. Cleve [Cle86] famously showed that the block builder and the proposer cannot successfully perform a fair exchange without additional help. In practice, the two parties are currently relying on a trusted intermediary, known as the relay.

A recent proposal by Noyes and Policharla [NP24] suggests replacing the trusted relay by a distributed network of parties. The block builder uses silent threshold encryption to encrypt their data towards the

⁵ Constant here means independent of n , but of course linear in the security parameter when measured in bits.

⁶ While recipient keys are small, the construction of Reyzin, Smith and Yakubov assumes that the sender has a public key of size $\text{poly}(n)$, where n is the number of recipients.

parties in the distributed network. Once the proposer publishes their data, the parties in the distributed network start publishing decryption shares, thereby revealing the builder’s data.

The distributed network envisioned by Noyes and Policharla contains hundreds of parties and the authors assume that some fraction $< 1/2$ of the network participants is corrupt. For instance, assuming at most $n/3$ nodes among the n nodes in the distributed network are corrupt is sufficient for our silent threshold encryption scheme. Since parties may arbitrarily join and leave the network, one cannot assume a distributed key generation and thus we require the silent setup that silent threshold encryption provides. Since some of the parties in the distributed network may be malicious, we require reconstruction to succeed even if the adversary provides malicious decryption shares in an attempt to disrupt the decryption process. Replacing the scheme Garg et al. [GKP+24] in the proposal of Noyes and Policharla with ours would result in significantly smaller public keys for all network nodes and stronger security guarantees at the cost of a modest increase in the ciphertext size.

1.3 Application: Encrypted Mempools with Small On-Chain Footprint

In the Ethereum blockchain, submitted user transactions are first stored in a public distributed mempool before being executed and stored on-chain. Users submit their transactions to the mempool in the clear, meaning that all transactions data is publicly visible to everybody on the network. As shown by Daian et al. [DGK+20], this allows for financial manipulations, enabling sophisticated actors to submit correlated transactions that extract money from honest users.

To avoid such financial manipulation, one can let each user submit their transactions in encrypted form, only decrypting them when they are being executed. This approach directly raises two questions. Under which encryption key is the data encrypted and where is the ciphertext stored? Directly storing large ciphertexts “on the blockchain” is wasteful, since on-chain storage is a scarce resource. Using our silent threshold encryption, a user can pick an ephemeral symmetric key, encrypt the data under this key, and encrypt the ephemeral key itself using threshold encryption towards the network participants. The small threshold encrypted key and the symmetric ciphertext can be submitted to the mempool, but the larger threshold encrypted ciphertext can be kept off-chain⁷. When the encrypted ephemeral key makes its way onto the chain (either via the user or via threshold decryption), the corresponding transaction becomes accessible. Using our silent threshold encryption scheme, we allow for small symmetric ciphertexts encrypting the ephemeral key and do not require the network participants to run any interactive key generation procedures. Also, in contrast to earlier constructions of silent threshold encryption, we avoid large hints or public keys. Such large keys are prohibitive in a blockchain setting, as they would need to be stored permanently on-chain.

2 Preliminaries

We denote the computational security parameter by λ and the statistical security parameter by κ . We assume that $\lambda/\kappa \in \Theta(1)$. We assume that all algorithms implicitly get 1^λ (the unary representation of λ) implicitly as input. We use standard cryptographic terminology like PPT and negligible functions. Throughout, we let $\text{negl}(\lambda)$ be an anonymous negligible function in λ . We write $\lg x$ to denote the logarithm base 2 of x . We use $\subset$ to denote a (not necessarily strict) subset. We use the notation $\leftarrow$ to denote sampling from distributions (e.g., randomized algorithms), and sampling uniformly from a finite set. Unless specified otherwise, adversaries are implicitly stateful. We denote by Shuffle an algorithm that takes as input a sequence of elements and outputs the same elements shuffled according to a permutation sampled uniformly at random.

2.1 Secret Sharing

We make use of standard threshold secret sharing schemes, where a party can share a message msg running a Share algorithm. As a result, one obtains n shares $s_1, \dots, s_n$, and any set of sufficiently many shares allows to reconstruct msg via algorithm Rec . We define both correctness and (perfect) privacy below. A classical instantiation of this is Shamir’s secret sharing [Sha79].

⁷ In our high-level problem description we ignore the issue of how off-chain data is reliably stored, but note that solutions like data availability network layers [ASB+21; HSW24] exist.

Definition 1 (Secret Sharing). A k -out-of- n threshold secret sharing scheme $SS = (\text{Share}, \text{Rec})$ with message space $\mathcal{M}$ and share space $\mathcal{S}$ is a tuple of PPT algorithms, which are defined as follows:

$\text{Share}(\text{msg}) \rightarrow (s_1, \dots, s_n)$: The sharing algorithm takes message $\text{msg} \in \mathcal{M}$ as input and returns a set of shares $s_1, \dots, s_n \in \mathcal{S}$.

$\text{Rec}(\{s_i\}_{i \in I}) \rightarrow \text{msg}$: The reconstruction algorithm takes a set of shares $s_i \in \mathcal{S}$ as input and returns message $\text{msg} \in \mathcal{M} \cup \{\perp\}$.

Without loss of generality, we assume that all shares output by Share are distinct.

Definition 2 (Correctness). A k -out-of- n threshold secret sharing scheme $SS = (\text{Share}, \text{Rec})$ with message space $\mathcal{M}$ is correct, if for any $\text{msg} \in \mathcal{M}$ and any $I \subset [n]$ with $|I| \geq k$, it holds that

$$\Pr [\text{Rec}(\{s_i\}_{i \in I}) = \text{msg} : (s_1, \dots, s_n) \leftarrow \text{Share}(\text{msg})] = 1.$$

Definition 3 (Privacy). A k -out-of- n threshold secret sharing scheme $SS = (\text{Share}, \text{Rec})$ with message space $\mathcal{M}$ is perfectly private, if for any adversary $\mathcal{A}$, any two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}$ and any $I \subset [n]$ with $|I| = k - 1$, it holds that

$$\Pr \left[\begin{array}{l} b' = b : (s_1, \dots, s_n) \leftarrow \text{Share}(\text{msg}_b) \\ b' \leftarrow \mathcal{A}(\{s_i\}_{i \in I}) \end{array} \right] = \frac{1}{2}.$$

2.2 Secret-Key Encryption

We define secret-key encryption and security for it.

Definition 4 (Secret-Key Encryption). A secret-key encryption scheme $SKE = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$, key space $\mathcal{K}$, and ciphertext space $\mathcal{C}$ is a tuple of PPT algorithms, which are defined as follows:

$\text{Gen}(1^\lambda) \rightarrow k$: The key generation algorithm takes security parameter 1^λ as input and returns key $k \in \mathcal{K}$.

$\text{Enc}(k, \text{msg}) \rightarrow \text{ctxt}$: The encryption algorithm takes key $k \in \mathcal{K}$ and message $\text{msg} \in \mathcal{M}$ as input and returns ciphertext $\text{ctxt} \in \mathcal{C}$.

$\text{Dec}(k, \text{ctxt}) \rightarrow \text{msg}$: The deterministic decryption algorithm takes key $k \in \mathcal{K}$ and ciphertext $\text{ctxt} \in \mathcal{C}$ as input and returns a message $\text{msg} \in \mathcal{M} \cup \{\perp\}$.

Definition 5 (Correctness). Secret-key encryption scheme $SKE = (\text{Gen}, \text{Enc}, \text{Dec})$ is correct, if for any PPT adversary $\mathcal{A}$, it holds that

$$\Pr \left[\begin{array}{l} \text{Dec}(k, \text{ctxt}) \neq \text{msg} : \\ \quad k \leftarrow \text{Gen}(1^\lambda) \\ \quad \text{msg} \leftarrow \mathcal{A}(k) \\ \quad \text{ctxt} \leftarrow \text{Enc}(k, \text{msg}) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 6 (Semantic Security). Secret-key encryption scheme $SKE = (\text{Gen}, \text{Enc}, \text{Dec})$ is semantically secure, if for any PPT adversary $\mathcal{A}$, it holds that

$$\Pr \left[\begin{array}{l} b = b' : \\ \quad k \leftarrow \text{Gen}(1^\lambda), b \leftarrow \{0, 1\} \\ \quad (\text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}(1^\lambda) \\ \quad \text{ctxt} \leftarrow \text{Enc}(k, \text{msg}_b) \\ \quad b' \leftarrow \mathcal{A}(\text{ctxt}) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

2.3 Public-Key Encryption

We make use of public-key encryption, as we define it next. The security notion that we consider is a CCA1-variant of *indistinguishability of keys*, as defined by Bellare et al. [BBD+01]. Their work also shows that Cramer-Shoup encryption [CS98] satisfies the stronger CCA2 variant of this notion, under the Decisional Diffie-Hellman assumption. One can also see that hashed ElGamal encryption satisfies this notion under the gap-Computational Diffie Hellman assumption [KL14]. For a post-quantum instantiation, Kyber can be used [MX23].

Definition 7 (Public-Key Encryption). A public-key encryption scheme $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ is a tuple of PPT algorithms, which are defined as follows:

$\text{Gen}(1^\lambda) \rightarrow (\text{ek}, \text{dk})$: The key generation algorithm takes security parameter 1^λ as input and returns key pair (ek, dk) .

$\text{Enc}(\text{ek}, \text{msg}) \rightarrow \text{ctxt}$: The encryption algorithm takes encryption key ek and message $\text{msg} \in \mathcal{M}$ as input and returns ciphertext $\text{ctxt} \in \mathcal{C}$.

$\text{Dec}(\text{dk}, \text{ctxt}) \rightarrow \text{msg}$: The deterministic decryption algorithm takes decryption key dk and ciphertext $\text{ctxt} \in \mathcal{C}$ as input and returns a message $\text{msg} \in \mathcal{M} \cup \{\perp\}$.

Definition 8 (Correctness). Public-key encryption scheme $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ is correct, if for any PPT adversary $\mathcal{A}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{ek}, \text{dk}) \leftarrow \text{Gen}(1^\lambda) \\ \text{Dec}(\text{dk}, \text{ctxt}) \neq \text{msg} : \begin{array}{l} \text{msg} \leftarrow \mathcal{A}(\text{ek}) \\ \text{ctxt} \leftarrow \text{Enc}(\text{ek}, \text{msg}) \end{array} \end{array} \right] \leq \text{negl}(\lambda).$$

$\text{Exp}_{\text{PKE}, \mathcal{A}}^{\text{IK-CCA1}}(1^\lambda)$	$\text{Dec0}(b, \text{ctxt})$
1 : $(\text{ek}_0, \text{dk}_0) \leftarrow \text{Gen}(1^\lambda), (\text{ek}_1, \text{dk}_1) \leftarrow \text{Gen}(1^\lambda)$	1 : return $\text{Dec}(\text{dk}_b, \text{ctxt})$
2 : $(\text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Dec0}}(\text{ek}_0, \text{ek}_1)$	
3 : $b \leftarrow \{0, 1\}, \text{ctxt} \leftarrow \text{Enc}(\text{ek}_b, \text{msg}_b)$	
4 : $b' \leftarrow \mathcal{A}(\text{ctxt})$	
5 : return $(b = b')$	

Fig. 1. Indistinguishability of keys for public key encryption.

Definition 9 (Indistinguishability of Keys). Public-key encryption scheme $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ has indistinguishable keys, if for any PPT adversary $\mathcal{A}$, it holds that

$$\Pr \left[\text{Exp}_{\text{PKE}, \mathcal{A}}^{\text{IK-CCA1}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $\text{Exp}_{\text{PKE}, \mathcal{A}}^{\text{IK-CCA1}}$ is defined in Figure 1.

2.4 Non-Interactive Key Exchange

We define non-interactive key exchange, following [CKS08; FHK+13; Lan23]. First, we recall the syntax. Then, we give a security notion that is tailored to our use case. One can easily see that this new security notion is implied by the *CKS-heavy* notion in the terminology of [FHK+13]. Efficient constructions satisfying this notion can be found in [CKS08; FHK+13], e.g., relying on the Computational Diffie-Hellman assumption.

Definition 10 (Non-Interactive Key Exchange). A non-interactive key exchange (NIKE) scheme $\text{NIKE} = (\text{Gen}, \text{SharedKey})$ with identity space $\mathcal{ID}$, public key space $\mathcal{PK}$, and key space $\mathcal{K}$ is a pair of PPT algorithms, which are defined as follows:

$\text{Gen}(1^\lambda, \text{id}) \rightarrow (\text{pk}, \text{sk})$: The key generation algorithm takes security parameter 1^λ and an identity $\text{id} \in \mathcal{ID}$ as input and returns key pair (pk, sk) , where $\text{pk} \in \mathcal{PK}$.

$\text{SharedKey}(\text{pk}_1, \text{id}_1, \text{sk}_2, \text{id}_2) \rightarrow K$: The shared key derivation algorithm is deterministic and takes as input a public key pk_1 with identity id_1 , and a secret key sk_2 with identity id_2 and returns a shared key $K \in \mathcal{K}$.

Definition 11 (Correctness). Non-interactive key exchange scheme $\text{NIKE} = (\text{Gen}, \text{SharedKey})$ with identity space $\mathcal{ID}$ is correct, if for any pair of identities $\text{id}_1, \text{id}_2 \in \mathcal{ID}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pk}_1, \text{sk}_1) \leftarrow \text{Gen}(1^\lambda, \text{id}_1) \\ (\text{pk}_2, \text{sk}_2) \leftarrow \text{Gen}(1^\lambda, \text{id}_2) \\ K_1 := \text{SharedKey}(\text{pk}_1, \text{id}_1, \text{sk}_2, \text{id}_2) \\ K_2 := \text{SharedKey}(\text{pk}_2, \text{id}_2, \text{sk}_1, \text{id}_1) \end{array} : K_1 \neq K_2 \right] \leq \text{negl}(\lambda).$$

Definition 12 (One-Pair Malicious-Key Security). Non-interactive key exchange scheme $\text{NIKE} = (\text{Gen}, \text{SharedKey})$ with identity space $\mathcal{ID}$ and key space $\mathcal{K}$ is one-pair malicious-key secure, if for all PPT adversaries $\mathcal{A}$, it holds that

$$\Pr \left[\text{Exp}_{\text{NIKE}, \mathcal{A}}^{\text{OP-MK}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $\text{Exp}_{\text{NIKE}, \mathcal{A}}^{\text{OP-MK}}$ is defined in Figure 2.

$\text{Exp}_{\text{NIKE}, \mathcal{A}}^{\text{OP-MK}}(1^\lambda)$

```

1 :  $(\text{id}_1^*, \text{id}_2^*) \leftarrow \mathcal{A}(1^\lambda)$ 
2 :  $(\text{pk}_1^*, \text{sk}_1^*) \leftarrow \text{Gen}(1^\lambda, \text{id}_1^*)$ 
3 :  $(\text{id}_j, \text{pk}_j)_{j=1}^k \leftarrow \mathcal{A}^{\text{SharedKey}(\cdot, \cdot, \text{sk}_1^*, \text{id}_1^*)}(\text{pk}_1^*)$ 
4 :  $(\text{pk}_2^*, \text{sk}_2^*) \leftarrow \text{Gen}(1^\lambda, \text{id}_2^*)$ 
5 : for  $j \in [k]$ :  $K_j := \text{SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}_2^*, \text{id}_2^*)$ 
6 :  $b \leftarrow \{0, 1\}$ 
7 : if  $b = 0$ :  $K^* := \text{SharedKey}(\text{pk}_1^*, \text{id}_1^*, \text{sk}_2^*, \text{id}_2^*)$  else  $K^* \leftarrow \mathcal{K}$ 
8 :  $b' \leftarrow \mathcal{A}(\text{pk}_2^*, K^*, (K_j)_{j=1}^k)$ 
9 : if  $|\{\text{id}_j\}_{j=1}^k \cup \{\text{id}_1^*, \text{id}_2^*\}| < k + 2$ : return 0    // id's must be distinct
10: if  $b = b'$ : return 1 else: return 0

```

Fig. 2. One-pair malicious-key security for non-interactive key exchange.

2.5 Static Dictionaries

A dictionary is a randomized data structure that allows for storing key/value pairs and allows for looking up the value corresponding to some given search key efficiently. A *static* dictionary requires all key/value pairs to be inserted at once. It is well known how to construct highly efficient static dictionaries, whose size linearly depends on the input set size and where each lookup operation only requires a small constant number of word operations [FKS84].

Definition 13 (Static Dictionary). Let $\kappa \in \mathbb{N}$ and $k, v \in \text{poly}(\kappa)$. A static dictionary with key space $\{0, 1\}^k$, value space $\{0, 1\}^v$, randomness space $\mathcal{R}$, and dictionary space $\mathcal{DI}$ is a pair of PPT algorithms $\text{DS} = (\text{Insert}, \text{Get})$ which are defined as follows:

$\text{Insert}(S, R) \rightarrow D$: The insertion algorithm takes a set of key/value pairs $S \subseteq \{0, 1\}^k \times \{0, 1\}^v$ and a random tape $R \in \mathcal{R}$ as input and returns a dictionary D .

$\text{Get}(D, R, x) \rightarrow y$: The lookup algorithm takes dictionary D , random tape R and key $x \in \{0, 1\}^k$ as input and returns value $y \in \{0, 1\}^v \cup \{\perp\}$.

Definition 14 (Correctness). A static dictionary $\text{DS} = (\text{Insert}, \text{Get})$ with key space $\{0, 1\}^k$, value space $\{0, 1\}^v$ and randomness space $\mathcal{R}$ is correct, if for all sets $S \subset \{0, 1\}^k \times \{0, 1\}^v$ of pairs with pairwise distinct keys with $|S| \leq \text{poly}(\kappa)$ and $x \in \{0, 1\}^k$, it holds that

$$\Pr \left[\text{Get}(\text{Insert}(S, R), R, x) = \begin{cases} y & \text{if } (x, y) \in S \\ \perp & \text{if for all } (x', y') \in S : x' \neq x \end{cases} \right] \geq 1 - \text{negl}(\kappa),$$

where the probability is taken over the random choice of $R \in \mathcal{R}$.

3 Silent Threshold Encryption

In this section, we define and construct silent threshold encryption. We state the formal definitions for threshold encryption and our new notion of *one-shot* adaptive security in [Section 3.1](#). We then provide an intuitive description of our construction in [Section 3.2](#) and provide a formal description along with its security analysis in [Section 3.3](#).

3.1 Definition

Recall that a threshold encryption scheme allows for encrypting a messages towards a group of users, such that sufficiently large coalitions can decrypt the message, but coalitions that are too small cannot. We consider the silent setting, where each user locally generates their own key material. Furthermore, we focus on the non-interactive decryption setting in which each user independently provides a decryption share and the message can be recovered by anybody who has access to the ciphertext and sufficiently many decryption shares. Lastly, we assume that every user has a unique identifier and that their generated key material is associated with this identifier. We note that these identifiers do not need to have any specific structure and we only require them to be distinct. In the context of Ethereum, these could, for example, just be the validators' keys.

Definition 15 ((t, n)-Silent Threshold Encryption). A (t, n)-silent threshold encryption scheme $\text{STE} = (\text{Gen}, \text{Enc}, \text{PartDec}, \text{Dec})$ with message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and identity space $\mathcal{ID}$ is a tuple of PPT algorithms, which are defined as follows:

$\text{Gen}(1^\lambda, \text{id}) \rightarrow (\text{ek}, \text{dk})$: The key generation algorithm takes security parameter λ and identity $\text{id} \in \mathcal{ID}$ as input and returns key pair (ek, dk) .

$\text{Enc}((\text{ek}_1, \text{id}_1), \dots, (\text{ek}_n, \text{id}_n), \text{msg}) \rightarrow \text{ctxt}$: The encryption algorithm takes encryption keys $\text{ek}_1, \dots, \text{ek}_n$ with associated identities $\text{id}_1, \dots, \text{id}_n \in \mathcal{ID}$ and message $\text{msg} \in \mathcal{M}$ as input and returns ciphertext $\text{ctxt} \in \mathcal{C}$.

$\text{PartDec}(\text{dk}, \text{id}, \text{ctxt}) \rightarrow \text{pdec}$: The deterministic partial decryption algorithm takes decryption key dk , identity $\text{id} \in \mathcal{ID}$ and ciphertext $\text{ctxt} \in \mathcal{C}$ as input and returns partial decryption pdec .

$\text{Dec}(\text{ctxt}, \text{pdec}_1, \dots, \text{pdec}_n) \rightarrow \text{msg}$: The deterministic decryption algorithm takes ciphertext $\text{ctxt} \in \mathcal{C}$ and partial decryption $\text{pdec}_1, \dots, \text{pdec}_n$ as input and returns message $\text{msg} \in \mathcal{M} \cup \{\perp\}$.

Informally, we want the scheme to satisfy correctness, meaning that if a message msg is encrypted to a ciphertext ctxt , then given a sufficient number of valid decryption shares it should be possible to recover msg from ctxt using the decryption algorithm Dec .

In the following, we define a stronger notion called *robustness*, which implies correctness. Robustness ensures that even in the presence of an adversary who controls up to t parties, the decryption process still yields the correct result. The adversary can corrupt some parties at the start of the experiment, and

others adaptively via a corruption oracle. For the parties corrupted at the start, the adversary may choose their keys and their identities. It also has access to a decryption oracle, allowing it to learn decryption shares. The adversary’s goal is to cause decryption to fail. Specifically, the experiment proceeds as follows: First, the adversary selects a message msg , which is then honestly encrypted into a ciphertext ctxt and given to the adversary. The adversary may then perform a final round of corruptions, adaptively based on the ciphertext, but all such corruptions occur simultaneously (hence the name *one-shot*). It then provides decryption shares on behalf of the corrupted parties. These are combined with honest shares from the remaining parties to attempt decryption of ctxt . The adversary wins if the decryption process fails, i.e., if the output is not equal to msg .

Definition 16 (Robustness Under One-Shot Adaptive Corruptions). A (t, n) -silent threshold encryption scheme $\text{STE} = (\text{Gen}, \text{Enc}, \text{PartDec}, \text{Dec})$ is robust under one-shot adaptive corruptions, if for all PPT adversaries $\mathcal{A}$ and all tuples of identifiers $\text{id} = (\text{id}_1, \dots, \text{id}_n) \in \mathcal{ID}$, it holds that

$$\Pr \left[\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-AR}}(1^\lambda) = 1 \right] \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-AR}}$ is defined in Figure 3.

The second security experiment we define for a silent threshold encryption scheme models *privacy*. As before, we define the game in a one-shot adaptive setting. In this experiment, the adversary again has access to both a partial decryption oracle and a corruption oracle. It begins by choosing two messages. One of them is selected uniformly at random and encrypted into a challenge ciphertext ctxt , which is then given to the adversary. After receiving ctxt , the adversary may perform a final round of corruptions, which are executed simultaneously in the one-shot model. It obtains the decryption keys corresponding to the corrupted parties and must then guess which of the two messages was encrypted. The adversary is restricted to corrupting at most t parties in total.

Definition 17 (Privacy Under One-Shot Adaptive Corruptions). A (t, n) -silent threshold encryption scheme $\text{STE} = (\text{Gen}, \text{Enc}, \text{PartDec}, \text{Dec})$ satisfies privacy (CCA1) under one-shot adaptive corruptions, if for all PPT adversaries $\mathcal{A}$ and all tuples of identifiers $\text{id} = (\text{id}_1, \dots, \text{id}_n) \in \mathcal{ID}$, it holds that

$$\Pr \left[\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-CCA1}}(1^\lambda) = 1 \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-CCA1}}$ is defined in Figure 3.

3.2 Construction Overview

Before presenting the formal construction, we first provide an overview of the main design choices.

A Simple Idea. To begin, let us only consider static, rather than adaptive corruptions. That is, we assume that the set of corrupted users is fixed at the start of the protocol. Towards constructing a silent threshold encryption scheme, we let each user $i \in [n]$ generate a key pair $(\text{ek}_i, \text{dk}_i)$ for a public-key encryption scheme. To encrypt a message msg , the sender secret-shares msg into shares $(s_1, \dots, s_n)$ and encrypts each share s_i separately under the public key ek_i . The resulting ciphertext is a vector of individual ciphertexts. It is clear that any sufficiently large subset of users can decrypt enough shares to reconstruct the message. Unfortunately, the ciphertext size of this approach scales linearly in n , which is too inefficient.

To address this issue, we instead sample a small random committee $C \subset [n]$ of size m during encryption and only encrypt towards the members of this committee. When at most $(1/2 - \epsilon)n$ users are corrupt, then choosing $m = \text{poly}(\epsilon)$ is sufficient to ensure that (with overwhelming probability) at least half the users in the committee are honest. Using the naive approach from above, we can now encrypt towards this committee with the secret sharing threshold being $m/2$. Importantly, we note that the size of the committee is *independent* of n .

$\text{Exp}_{\text{STE}, \mathcal{A}, t, id}^{\text{STE-CCA1}}(1^\lambda)$	$\text{Exp}_{\text{STE}, \mathcal{A}, t, id}^{\text{STE-AR}}(1^\lambda)$
1: $I \leftarrow \mathcal{A}(1^\lambda)$ 2: $\text{Corr} := I, \text{PartDecs} := \emptyset$ 3: for $i \in [n] \setminus \text{Corr}$: 4: $(ek_i, dk_i) \leftarrow \text{Gen}(1^\lambda, id_i)$ 5: $((ek_i, id_i)_{i \in I}, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((ek_i, id_i)_{i \in [n] \setminus \text{Corr}})$ 6: if $\exists i, j \in [n]$ s.t. $id_i = id_j \wedge i \neq j$: return 0 7: $b \leftarrow \{0, 1\}, \text{ctxt} \leftarrow \text{Enc}((ek_1, id_1), \dots, (ek_n, id_n), \text{msg}_b)$ 8: $S \leftarrow \mathcal{A}(\text{ctxt})$ 9: if $S \cap \text{Corr} \neq \emptyset$: return 0 10: $\text{Corr} := \text{Corr} \cup S$ 11: $b' \leftarrow \mathcal{A}(\{dk_i\}_{i \in S})$ 12: if $ \text{Corr} \leq t \wedge b = b'$: 13: return 1 14: else : return 0	1: $I \leftarrow \mathcal{A}(1^\lambda)$ 2: $\text{Corr} := I, \text{PartDecs} := \emptyset$ 3: for $i \in [n] \setminus \text{Corr}$: 4: $(ek_i, dk_i) \leftarrow \text{Gen}(1^\lambda, id_i)$ 5: $((ek_i, id_i)_{i \in I}, \text{msg}) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((ek_i, id_i)_{i \in [n] \setminus \text{Corr}})$ 6: if $\exists i, j \in [n]$ s.t. $id_i = id_j \wedge i \neq j$: return 0 7: $\text{ctxt} \leftarrow \text{Enc}((ek_1, id_1), \dots, (ek_n, id_n), \text{msg})$ 8: $S \leftarrow \mathcal{A}(\text{ctxt})$ 9: if $S \cap \text{Corr} \neq \emptyset$: return 0 10: $\text{Corr} := \text{Corr} \cup S$ 11: $(\text{pdec}_i)_{i \in \text{Corr} \cup S} \leftarrow \mathcal{A}(\{dk_i\}_{i \in S})$ 12: for $i \in [n] \setminus \text{Corr}$: $\text{pdec}_i \leftarrow \text{PartDec}(dk_i, \text{ctxt})$ 13: if $ \text{Corr} \leq t \wedge \text{Dec}(\text{ctxt}, \{\text{pdec}_i\}_{i \in [n]}) \neq \text{msg}$: return 1 14: else : return 0
$\text{Cor0}(i)$	$\text{Dec0}(\text{ctxt}, i)$
1: if $i \in \text{Corr}$: return $\perp$ 2: else : 3: $\text{Corr} := \text{Corr} \cup \{i\}$ 4: return dk_i	1: if $i \in \text{Corr}$: return $\perp$ 2: else : 3: $\text{PartDecs} := \text{PartDecs} \cup \{\text{ctxt}\}$ 4: return $\text{PartDec}(dk_i, id_i, \text{ctxt})$

Fig. 3. Privacy (left) and robustness (right) under one-shot adaptive corruptions for silent threshold encryption.

Adaptive Security. To achieve security against adaptive corruptions, we must prevent the adversary from choosing who to corrupt based on who is in the sampled committee. For this reason, we use an anonymous public-key encryption scheme, which ensures that ciphertexts do not reveal the identity of the recipient. Intuitively, this approach hides who is in each ciphertext’s committee and prevents the adversary from specifically targeting parties inside the committee corresponding to the challenge ciphertext. While this approach works intuitively, formally proving security turns out to be highly non-trivial and even proving one-shot adaptive security requires several additional tricks, like early-terminating hybrids and the use of (efficient) non-committing encryption. Despite not being able to prove it, we do conjecture that our construction should satisfy full adaptive security.

Adding Robustness. So far, we have only discussed correctness under the assumptions that all provided decryption shares are generated honestly. However, corrupt users may provide malformed decryption shares, thereby preventing the reconstruction of the message. To mitigate this, our construction ensures that decryption shares are verifiable. For this, we append a random value r_i to each share s_i and include tag $h_i = H(r_i \| s_i)$ in the ciphertext, where H is modeled as a random oracle. Upon receiving a decryption share (s_i, r_i) , we can verify that it is well formed by checking if it matches the corresponding tag in the ciphertext. Decryption succeeds as long as a sufficient number of valid shares are recovered, irrespective of how many malformed shares are additionally provided by the adversary.

Dealing with Large Messages. To allow for encrypting large messages efficiently, we use hybrid encryption. Instead of secret sharing the message, we secret share a short symmetric key k and threshold encrypt it using the scheme outlined above. Additionally, we add a symmetric-key encryption of the message under key k to the ciphertext. Encrypting a B -bit message, assuming that the anonymous public-key encryption scheme has ciphertexts of length $\mathcal{O}(\lambda)$, results in ciphertexts with bit length $\mathcal{O}(B + \lambda \cdot \text{poly}(\epsilon))$.

Reducing the Computational Complexity. During encryption and decryption, the computational complexity currently scales with $\Omega(\text{poly}(\epsilon))$. For low-latency use-cases, where encryption and decryption have to be highly efficient, this can be a practical bottleneck.

In the case of encryption, we note that the computationally expensive part is to threshold encrypt the ephemeral key k . Since this is independent of the message, the encrypting party can preprocess this

computation and then encrypt efficiently when the message arrives by performing a single symmetric-key encryption. Making decryption concretely fast is more tricky. Currently, each user needs to try and decrypt *each* individual ciphertext to determine if one of the ciphertexts belongs to them. For efficiency reasons, we would like each user to directly know which individual ciphertext in the ciphertext vector belongs to them. For adaptive security reasons, as mentioned above, we would like the ciphertext vector to not directly reveal which individual ciphertext belongs to which user.

To overcome this dilemma, we make use of static dictionaries. Let C be a sampled committee of users during encryption. A first naive approach is to define the set $S = \{(i, d_i) \mid i \in C\}$, where d_i is all the ciphertext data associated with user i , to insert S into a static dictionary, and to let that dictionary of encrypted data be the ciphertext. Upon seeing the ciphertext dictionary, each user i can use the dictionary's lookup operation to find their individual ciphertext efficiently. While this works from a correctness perspective, it does not work from a security perspective. The dictionary, along with the positions of the corrupt users' ciphertexts within it, may reveal information about which honest users are part of the committee. The reason for this issue is the fact that we used the plain users' identifiers as the keys during insertion into the dictionary.

Instead, our idea is to derive the keys for the dictionary using a non-interactive key exchange: we generate an additional ephemeral public key during encryption and append it to the ciphertext. Using the non-interactive key exchange between this ephemeral public key and the public key of each user in the committee C , we obtain a shared secret K_i between each user i in C and the encrypting party. Now rather than inserting pairs of the form (i, d_i) into the dictionary, we insert (K_i, d_i) . By the security properties of non-interactive key exchange, from the adversary's perspective, the dictionary's keys of all honest users are now uniformly random strings that do not depend on the users identities and thus using a dictionary does not harm security of our encryption scheme.

3.3 Construction

We require the following building blocks, for a parameter $m \in \mathbb{N}$:

- Random oracle $H: \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$;
- Secret-key encryption scheme $\text{SKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$, key space $\mathcal{K}_{\text{SKE}}$, and ciphertext space $\mathcal{C}_{\text{SKE}}$;
- $m/2$ -out-of- m threshold secret sharing scheme $\text{SS} = (\text{Share}, \text{Rec})$ with message space $\mathcal{K}$ and share space $\mathcal{S}$, where we assume that $\mathcal{S}$ can naturally be embedded into $\{0, 1\}^{2\lambda}$;
- Public-key encryption scheme $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\{0, 1\}^\lambda \times \mathcal{S}$ and ciphertext space $\mathcal{C}_{\text{PKE}}$;
- Non-interactive key exchange scheme $\text{NIKE} = (\text{Gen}, \text{SharedKey})$ with identity space $\mathcal{ID}$, public key space $\mathcal{PK}$, and key space $\mathcal{K}_{\text{NIKE}}$;
- Static dictionary $\text{DS} = (\text{Insert}, \text{Get})$ with key space $\mathcal{K}_{\text{NIKE}}$, value space $\mathcal{C}_{\text{PKE}} \times \{0, 1\}^{2\lambda} \times \{0, 1\}^{2\lambda}$, randomness space $\mathcal{R}$, and dictionary space $\mathcal{DI}$;

Our silent threshold encryption scheme STE also has message space $\mathcal{M}$ (same as SKE). It has ciphertext space

$$\mathcal{C} = \mathcal{C}_{\text{SKE}} \times \mathcal{DI} \times \mathcal{R} \times \mathcal{PK}.$$

We present the construction of STE in [Figure 4](#).

3.4 Security Analysis

We start the analysis of our scheme by showing a core lemma. Intuitively, it bounds the probability that an adversary corrupts at least half of the committee C associated with the challenge ciphertext. The proof relies on the security of NIKE and the anonymity of PKE . We will then use this core lemma to show robustness and privacy.

Lemma 1 (Core Lemma). *Fix n, t, ϵ with $\epsilon > 0$ and $t \leq (1/2 - \epsilon)n$. Fix $B = m/2$. Assume that $H: \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ is modeled as a random oracle. Assume that NIKE is one-pair malicious-key secure ([Definition 12](#)) and that PKE has indistinguishable keys ([Definition 9](#)), and consider STE as defined in*

Gen ($1^\lambda, \text{id}$)	PartDec ($\text{dk}, \text{id}, \text{ctxt}$)
1 : $(\widetilde{\text{ek}}, \widetilde{\text{dk}}) \leftarrow \text{PKE.Gen}(1^\lambda)$	1 : parse dk as $(\widetilde{\text{dk}}, \text{sk})$
2 : $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id})$	2 : parse ctxt as $(\widetilde{\text{ctxt}}, D, R, \text{pk})$
3 : $\text{ek} := (\widetilde{\text{ek}}, \text{pk}), \text{dk} := (\widetilde{\text{dk}}, \text{sk})$	3 : $K \leftarrow \text{NIKE.SharedKey}(\text{pk}, 0, \text{sk}, \text{id})$
4 : return (ek, dk)	4 : $\widetilde{\text{ctxt}} \parallel \widehat{h} \parallel \widehat{w} \leftarrow \text{Get}(D, R, K)$
Enc ((ek_1, id_1), ..., (ek_n, id_n), msg)	5 : if $\widetilde{\text{ctxt}} \parallel \widehat{h} \parallel \widehat{w} = \perp$: return $\perp$
1 : $k \leftarrow \text{SKE.Gen}(1^\lambda), \widetilde{\text{ctxt}} \leftarrow \text{SKE.Enc}(k, \text{msg})$	6 : $r \leftarrow \text{PKE.Dec}(\widetilde{\text{dk}}, \widetilde{\text{ctxt}})$
2 : $C \leftarrow \{S \subset [n] \mid S = m\}$	7 : $s := H(r) \oplus w$
3 : $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$	8 : return (K, r, s)
4 : $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$	Dec ($\text{ctxt}, \text{pdec}_1, \dots, \text{pdec}_n$)
5 : $E := \emptyset$	1 : parse ctxt as $(\widetilde{\text{ctxt}}, D, R, \text{pk})$
6 : for $j \in C$:	2 : $S := \emptyset$
7 : parse ek_j as $(\widetilde{\text{ek}}_j, \text{pk}_j)$	3 : for $i \in [n]$:
8 : $r_j \leftarrow \{0, 1\}^\lambda, w_j := H(r_j) \oplus s_j$	4 : parse pdec_i as (K, r, s)
9 : $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\widetilde{\text{ek}}_j, r_j)$	5 : $\widetilde{\text{ctxt}} \parallel \widehat{h} \parallel \widehat{w} \leftarrow \text{Get}(D, R, K)$
10 : $h_j := H(r_j \parallel s_j)$	6 : if $H(r \parallel s) = \widehat{h}$: $S := S \cup \{s\}$
11 : $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$	7 : if $ S < m/2$: return $\perp$
12 : $E := E \cup \{(K_j, \text{ctxt}_j \parallel h_j \parallel w_j)\}$	8 : $k \leftarrow \text{Rec}(S)$
13 : $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$	9 : $\text{msg} \leftarrow \text{SKE.Dec}(k, \widetilde{\text{ctxt}})$
14 : return $(\widetilde{\text{ctxt}}, D, R, \text{pk})$	10 : return msg

Fig. 4. Our silent threshold encryption construction $\text{STE} = (\text{Gen}, \text{Enc}, \text{PartDec}, \text{Dec})$.

Figure 4. Then, for any PPT algorithm $\mathcal{A}$, and all tuples of identifiers $\text{id} = (\text{id}_1, \dots, \text{id}_n) \in \mathcal{ID}$, it holds that

$$\Pr [\text{Exp}_{\text{STE}, \mathcal{A}, B, t, \text{id}}^{\text{core}}(1^\lambda) = 1] \leq \text{negl}(\lambda) + e^{-2m\epsilon^2},$$

where the experiment $\text{Exp}_{\text{STE}, \mathcal{A}, B, t, \text{id}}^{\text{core}}(1^\lambda)$ is defined in *Figure 5*.

Proof. We prove the core lemma via a sequence of hybrid experiments. The idea is to consider the committee C that is sampled for the challenge ciphertext, and first replace all ciphertexts ctxt_j for uncorrupted committee members j with ciphertexts for an unrelated encryption key ek' . In a second step, we replace all shared NIKE keys K_j for these members with random keys. At this point, we will be able to argue that the adversary learns nothing about uncorrupted committee members, and we will use a Chernoff bound to argue the bound.

$\mathcal{H}_{\text{Rep-0}}$. This is the experiment from the lemma. We provide it as full pseudocode, using the notation from the scheme in *Figure 6*. Note that we also sample some *unrelated* encryption key pair $(\text{ek}', \text{dk}') \leftarrow \text{PKE.Gen}(1^\lambda)$ at the onset of the experiment. This key pair is not used yet in this hybrid, but will be used in the following hybrids. By definition, we have

$$\Pr [\text{Exp}_{\text{STE}, \mathcal{A}, B, t, \text{id}}^{\text{core}}(1^\lambda) = 1] = \Pr [\mathcal{H}_{\text{Rep-0}} = 1].$$

$\mathcal{H}_{\text{Rep-1}}, \dots, \mathcal{H}_{\text{Rep-}n}$. For $\iota \in [n]$, hybrid $\mathcal{H}_{\text{Rep-}\iota}$ is the same as $\mathcal{H}_{\text{Rep-}(\iota-1)}$ except that if $\iota \in C \setminus \text{Corr}$ we produce the ciphertext ctxt_ι included in the challenge ciphertext for party ι using encryption key ek' and without using r_ι . Precisely, we produce it as $\text{ctxt}_\iota \leftarrow \text{PKE.Enc}(\text{ek}', 0^\iota)$, where 0^ι is some default message of the right length. All oracle queries are still answered using the honest keys. We define $R = \{1, \dots, \iota\}$ as the set of parties for which we have replaced the ciphertexts with ciphertexts encrypted under this key ek' . See *Figure 6* for a formal description of these hybrids.

One can see that indistinguishability of the two hybrids follows via a simple reduction to the indistinguishable keys (*Definition 9*) property of PKE. Namely, with this, we get for every $\iota \in \{1, \dots, n\}$:

$$|\Pr [\mathcal{H}_{\text{Rep-}\iota-1} = 1] - \Pr [\mathcal{H}_{\text{Rep-}\iota} = 1]| \leq \text{negl}(\lambda).$$

$\text{Exp}_{\text{STE}, \mathcal{A}, B, t, id}^{\text{core}}(1^\lambda)$

```

1:  $I \leftarrow \mathcal{A}(1^\lambda)$ 
2:  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
3: for  $i \in [n] \setminus \text{Corr}$  :  $(\text{ek}_i, \text{dk}_i) \leftarrow \text{Gen}(1^\lambda, \text{id}_i)$ 
4:  $((\text{ek}_i, \text{id}_i)_{i \in I}, \text{msg}) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 
5: if  $\exists i, j \in [n]$  s.t.  $\text{id}_i = \text{id}_j \wedge i \neq j$  : return 0
6:  $\text{ctxt} \leftarrow \text{Enc}((\text{ek}_1, \text{id}_1), \dots, (\text{ek}_n, \text{id}_n), \text{msg})$ , let  $C$  be as in execution of  $\text{Enc}$ 
7:  $S \leftarrow \mathcal{A}(\text{ctxt})$ 
8: if  $S \cap \text{Corr} \neq \emptyset$  : return 0
9:  $\text{Corr} := \text{Corr} \cup S$ 
10: return  $(|\text{Corr}| \leq t \wedge |C \cap \text{Corr}| \geq B)$ 

```

Fig. 5. The experiment in the core lemma (Lemma 1). Oracles Cor0 and Dec0 are as in Figure 3.

$\mathcal{H}_{\text{Rep-W}}$. In this hybrid, we sample $w_j \leftarrow \{0, 1\}^{2\lambda}$ and $h_j \leftarrow \{0, 1\}^{2\lambda}$ at random instead of defining it as $w_j := H(r_j) \oplus s_j$ and $h_j := H(r_j \| s_j)$, for each $j \in C \setminus \text{Corr}$. The pseudocode is given in Figure 7. Note that the adversary can only see this change if it queries $H(r_j \| s_j)$ or $H(r_j)$, which happens with negligible probability as r_j has high entropy and as we removed every information about r_j via the previous hybrids. We get

$$|\Pr[\mathcal{H}_{\text{Rep-n}} = 1] - \Pr[\mathcal{H}_{\text{Rep-W}} = 1]| \leq \text{negl}(\lambda).$$

$\mathcal{H}_{\text{Rep-NIKE-0}}, \dots, \mathcal{H}_{\text{Rep-NIKE-n}}$. The hybrid $\mathcal{H}_{\text{Rep-NIKE-0}}$ is the same as $\mathcal{H}_{\text{Rep-W}}$. Further, for any $\iota \in [n]$, $\mathcal{H}_{\text{Rep-NIKE-}\iota}$ is the same as $\mathcal{H}_{\text{Rep-NIKE-}\iota-1}$, except that we replace the shared NIKE key K_ι in the challenge ciphertext with a random key, conditioned on ι being in the committee and uncorrupted. More precisely, we have a set $R' = \{1, \dots, \iota\}$, and if $j \in R'$ and $j \in C$ but $j \notin \text{Corr}$, then we sample $K_j \leftarrow \mathcal{K}$, whereas we keep K_j as before for all other $j \in C$. We do not change the oracles Cor0 and Dec0 . The detailed pseudocode is given in Figure 7. Now, from the definition, it is clear that

$$\Pr[\mathcal{H}_{\text{Rep-W}} = 1] = \Pr[\mathcal{H}_{\text{Rep-NIKE-0}} = 1].$$

Further, we can argue the indistinguishability of two subsequent hybrids $\mathcal{H}_{\text{Rep-NIKE-}\iota}$ and $\mathcal{H}_{\text{Rep-NIKE-}\iota-1}$ as follows. First, conditioned on the event that ι is corrupted via the corruption oracle or is not in the committee, the two hybrids are the same. So, condition on ι being uncorrupted and in the committee. We will sketch a reduction to the one-pair malicious-key security of NIKE (Definition 12) to bound the difference between the two hybrids in this case. The reduction simulates $\mathcal{H}_{\text{Rep-NIKE-}\iota}$ (resp. $\mathcal{H}_{\text{Rep-NIKE-}\iota-1}$) for the adversary, and aborts if at any point in time party ι becomes corrupted, or if party ι is not in the committee. It first outputs $\text{id}_1^* = \text{id}_\iota, \text{id}_2^* = 0$ to the game, and gets back pk_1^* , which it sets as $\text{pk}_\iota := \text{pk}_1^*$. It samples all remaining keys and the rest of party ι 's key honestly (in particular, it knows the decryption key for PKE). For any partial decryption query for party ι (which are made before the challenge ciphertext), the reduction uses its shared key oracle and the decryption key. When the challenge ciphertext needs to be constructed, the reduction outputs the identities and NIKE public keys of all parties except party ι , and gets back a public key pk_2^* , a challenge key K^* , and shared NIKE keys $\bar{K}_j$ between pk_2^* and the parties j except party ι . It uses pk_2^* as the key in the challenge ciphertext, and defines the shared keys K_j used to construct the challenge ciphertext as follows:

- If $j < \iota$, it samples K_j at random.
- If $j = \iota$, it sets K_j equal to the challenge key K^* .
- If $j > \iota$, it sets K_j equal to $\bar{K}_j$.

With this, it constructs the challenge ciphertext. Finally, the reduction outputs whatever the hybrid outputs. One can see that the reduction is efficient and perfectly simulates $\mathcal{H}_{\text{Rep-NIKE-}\iota-1}$ if $b = 0$ and $\mathcal{H}_{\text{Rep-NIKE-}\iota}$ if $b = 1$, where b is the bit in the one-pair malicious-key security game. With that, we get

$\mathcal{H}_{\text{Rep-}\iota}$	$\text{Dec0}(\text{ctxt}, i)$
<pre> 1: $R := \{1, \dots, \iota\}$ // Empty for $\iota = 0$ 2: $(\text{ek}', \text{dk}') \leftarrow \text{PKE.Gen}(1^\lambda)$ 3: $I \leftarrow \mathcal{A}(1^\lambda)$ 4: $\text{Corr} := I, \text{PartDecs} := \emptyset$ 5: for $i \in [n] \setminus \text{Corr}$: 6: $(\widetilde{\text{ek}}_i, \widetilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 7: $\text{ek}_i := (\widetilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\widetilde{\text{dk}}_i, \text{sk}_i)$ 8: $((\text{ek}_i = (\widetilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}) \leftarrow \mathcal{A}^{\text{Corr0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 9: if $\exists i, j \in [n]$ s.t. $\text{id}_i = \text{id}_j \wedge i \neq j$: return 0 10: $C \leftarrow \{S \subset [n] \mid S = m\}$ 11: $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 12: $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 13: $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg})$ 14: $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 15: $E := \emptyset$ 16: for $j \in C$: 17: $r_j \leftarrow \{0, 1\}^\lambda$ 18: if $j \in R \setminus \text{Corr}$: 19: $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\text{ek}', 0^l), w_j := H(r_j) \oplus s_j$ 20: else : 21: $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\widetilde{\text{ek}}_j, r_j), w_j := H(r_j) \oplus s_j$ 22: $h_j := H(r_j \ s_j)$ 23: $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 24: $E := E \cup \{(K_j, \text{ctxt}_j \ h_j \ w_j)\}$ 25: $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 26: $\text{ctxt} := (\text{ctxt}, D, R, \text{pk})$ 27: $S \leftarrow \mathcal{A}(\text{ctxt})$ 28: if $S \cap \text{Corr} \neq \emptyset$: return 0 29: $\text{Corr} := \text{Corr} \cup S$ 30: return $(\text{Corr} \leq t \wedge C \cap \text{Corr} \geq B)$ </pre>	<pre> 1: if $i \in \text{Corr}$: return $\perp$ 2: else : 3: $\text{PartDecs} := \text{PartDecs} \cup \{\text{ctxt}\}$ 4: parse ctxt as $(\widehat{\text{ctxt}}, D, R, \text{pk})$ 5: $K \leftarrow \text{NIKE.SharedKey}(\text{pk}, 0, \text{sk}_i, \text{id}_i)$ 6: $\widehat{\text{ctxt}} \ \widehat{h} \ \widehat{w} \leftarrow \text{Get}(D, R, K)$ 7: if $\widehat{\text{ctxt}} \ \widehat{h} \ \widehat{w} = \perp$: 8: return $\perp$ 9: $r \leftarrow \text{PKE.Dec}(\widetilde{\text{dk}}_i, \widehat{\text{ctxt}})$ 10: $s := H(r) \oplus w$ 11: return (K, r, s) </pre>
	<pre> Corr0(i) 1: if $i \in \text{Corr}$: return $\perp$ 2: else : 3: $\text{Corr} := \text{Corr} \cup \{i\}$ 4: return dk_i </pre>

Fig. 6. Hybrids $\mathcal{H}_{\text{Rep-}\iota}$ in the proof of the core lemma (Lemma 1).

$\mathcal{H}_{\text{Rep-W}}$

```

1:  $R := \{1, \dots, n\}$ 
2:  $(ek', dk') \leftarrow \text{PKE.Gen}(1^\lambda)$ 
3:  $I \leftarrow \mathcal{A}(1^\lambda)$ 
4:  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
5: for  $i \in [n] \setminus \text{Corr}$  :
6:    $(\tilde{ek}_i, \tilde{dk}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (pk_i, sk_i) \leftarrow \text{NIKE.Gen}(1^\lambda, id_i)$ 
7:    $ek_i := (\tilde{ek}_i, pk_i), dk_i := (\tilde{dk}_i, sk_i)$ 
8:    $((ek_i = (\tilde{ek}_i, pk_i), id_i)_{i \in I}, \text{msg}) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((ek_i, id_i)_{i \in [n] \setminus \text{Corr}})$ 
9:   if  $\exists i, j \in [n]$  s.t.  $id_i = id_j \wedge i \neq j$  : return 0
10:   $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
11:   $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 
12:   $(pk, sk) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
13:   $\widetilde{\text{ctxt}} \leftarrow \text{SKE.Enc}(k, \text{msg})$ 
14:   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
15:   $E := \emptyset$ 
16:  for  $j \in C$  :
17:     $r_j \leftarrow \{0, 1\}^\lambda$ 
18:    if  $j \in R \setminus \text{Corr}$  :
19:       $\text{ctxt}_j \leftarrow \text{PKE.Enc}(ek', 0^l), w_j, h_j \leftarrow \{0, 1\}^{2\lambda}$ 
20:    else :
21:       $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{ek}_j, r_j), w_j := H(r_j) \oplus s_j$ 
22:       $h_j := H(r_j \| s_j)$ 
23:       $K_j \leftarrow \text{NIKE.SharedKey}(pk_j, id_j, sk, 0)$ 
24:       $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
25:   $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
26:   $\text{ctxt} := (\widetilde{\text{ctxt}}, D, R, pk)$ 
27:   $S \leftarrow \mathcal{A}(\text{ctxt})$ 
28:  if  $S \cap \text{Corr} \neq \emptyset$  : return 0
29:   $\text{Corr} := \text{Corr} \cup S$ 
30: return  $(|\text{Corr}| \leq t \wedge |C \cap \text{Corr}| \geq B)$ 

```

 $\mathcal{H}_{\text{Rep-NIKE-}\ell}$

```

1:  $R := \{1, \dots, n\}$ 
2:  $(ek', dk') \leftarrow \text{PKE.Gen}(1^\lambda)$ 
3:  $R' := \{1, \dots, \ell\}$  // Empty for  $\ell = 0$ 
4:  $I \leftarrow \mathcal{A}(1^\lambda)$ 
5:  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
6: for  $i \in [n] \setminus \text{Corr}$  :
7:    $(\tilde{ek}_i, \tilde{dk}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (pk_i, sk_i) \leftarrow \text{NIKE.Gen}(1^\lambda, id_i)$ 
8:    $ek_i := (\tilde{ek}_i, pk_i), dk_i := (\tilde{dk}_i, sk_i)$ 
9:    $((ek_i = (\tilde{ek}_i, pk_i), id_i)_{i \in I}, \text{msg}) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((ek_i, id_i)_{i \in [n] \setminus \text{Corr}})$ 
10:  if  $\exists i, j \in [n]$  s.t.  $id_i = id_j \wedge i \neq j$  : return 0
11:   $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
12:   $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 
13:   $(pk, sk) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
14:   $\widetilde{\text{ctxt}} \leftarrow \text{SKE.Enc}(k, \text{msg})$ 
15:   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
16:   $E := \emptyset$ 
17:  for  $j \in C$  :
18:     $r_j \leftarrow \{0, 1\}^\lambda$ 
19:    if  $j \in R \setminus \text{Corr}$  :
20:       $\text{ctxt}_j \leftarrow \text{PKE.Enc}(ek', 0^l), w_j, h_j \leftarrow \{0, 1\}^{2\lambda}$ 
21:    else :
22:       $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{ek}_j, r_j), w_j := H(r_j) \oplus s_j$ 
23:       $h_j := H(r_j \| s_j)$ 
24:      if  $j \in R' \setminus \text{Corr}$  :  $K_j \leftarrow \mathcal{K}$ 
25:      else :  $K_j \leftarrow \text{NIKE.SharedKey}(pk_j, id_j, sk, 0)$ 
26:       $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
27:   $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
28:   $\text{ctxt} := (\widetilde{\text{ctxt}}, D, R, pk)$ 
29:   $S \leftarrow \mathcal{A}(\text{ctxt})$ 
30:  if  $S \cap \text{Corr} \neq \emptyset$  : return 0
31:   $\text{Corr} := \text{Corr} \cup S$ 
32: return  $(|\text{Corr}| \leq t \wedge |C \cap \text{Corr}| \geq B)$ 

```

Fig. 7. Hybrid $\mathcal{H}_{\text{Rep-W}}$ (left) and hybrids $\mathcal{H}_{\text{Rep-NIKE-}j}$ (right) in the proof of the core lemma (Lemma 1). Oracles Cor0 and Dec0 are as in Figure 6.

$\mathcal{H}_{\text{Resample}}$

```

1 :  $(\text{ek}', \text{dk}') \leftarrow \text{PKE.Gen}(1^\lambda)$ 
2 :  $I \leftarrow \mathcal{A}(1^\lambda)$ 
3 :  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
4 : for  $i \in [n] \setminus \text{Corr}$  :
5 :    $(\tilde{\text{ek}}_i, \tilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 
6 :    $\text{ek}_i := (\tilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\tilde{\text{dk}}_i, \text{sk}_i)$ 
7 :    $((\text{ek}_i = (\tilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 
8 :   if  $\exists i, j \in [n]$  s.t.  $\text{id}_i = \text{id}_j \wedge i \neq j$  : return 0
9 :    $\text{k} \leftarrow \text{SKE.Gen}(1^\lambda), \text{ctxt} \leftarrow \text{SKE.Enc}(\text{k}, \text{msg})$ 
10 :   $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
11 :   $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
12 :   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(\text{k}), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
13 :   $E := \emptyset$ 
14 :  for  $j \in C$  :
15 :     $r_j \leftarrow \{0, 1\}^\lambda$ 
16 :    if  $j \notin \text{Corr}$  :  $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\text{ek}', 0^l), w_j, h_j \leftarrow \{0, 1\}^{2\lambda}, K_j \leftarrow \mathcal{K}$ 
17 :    else :
18 :       $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_j, r_j), w_j := H(r_j) \oplus s_j$ 
19 :       $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 
20 :       $h_j := H(r_j \| s_j)$ 
21 :       $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
22 :       $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
23 :       $\text{ctxt} := (\text{ctxt}, D, R, \text{pk})$ 
24 :       $S \leftarrow \mathcal{A}(\text{ctxt})$ 
25 :      if  $S \cap \text{Corr} \neq \emptyset$  : return 0
26 :       $\text{Corr} := \text{Corr} \cup S$ 
27 :       $C' \leftarrow \{S \subset [n] \mid |S| = m\}$ 
28 :  return  $(|\text{Corr}| \leq t \wedge |C' \cap \text{Corr}| \geq B)$ 

```

Fig. 8. Hybrid $\mathcal{H}_{\text{Resample}}$ in the proof of the core lemma (Lemma 1). Oracles Cor0 and Dec0 are as in Figure 6.

that for every $\iota \in \{1, \dots, n\}$:

$$|\Pr[\mathcal{H}_{\text{Rep-NIKE-}\iota-1} = 1] - \Pr[\mathcal{H}_{\text{Rep-NIKE-}\iota} = 1]| \leq \text{negl}(\lambda).$$

$\mathcal{H}_{\text{Resample}}$. This hybrid is the same as $\mathcal{H}_{\text{Rep-NIKE-}n}$, except, after forming the challenge ciphertext, we resample the committee. That is, instead of checking $|C \cap \text{Corr}| > B$, we check that $|C' \cap \text{Corr}| > B$ for a newly sampled committee $C' \leftarrow \{S \subset [n] \mid |S| = m\}$, which is sampled after the adversary already terminated. We present the hybrid formally in Figure 8. Our claim is that this does not change the probability of the hybrid outputting 1. Namely, denote the corrupted set *before* the challenge ciphertext is given to the adversary by $\text{Corr}^{\text{Before}}$. Then, the reader may first observe the following:

- The committee C is independent of $\text{Corr}^{\text{Before}}$. This is because it is sampled after $\text{Corr}^{\text{Before}}$ is fixed. This also means that $C \cap \text{Corr}^{\text{Before}}$ and $C' \cap \text{Corr}^{\text{Before}}$ have the same distribution.
- Due to the changes that we have made in the previous hybrids, the set S is independent of $C \setminus \text{Corr}^{\text{Before}}$. This means that $(C \setminus \text{Corr}^{\text{Before}}) \cap S$ and $(C' \setminus \text{Corr}^{\text{Before}}) \cap S$ have the same distribution.

With this, and using the fact that S and $\text{Corr}^{\text{Before}}$ partition Corr , we get

$$\begin{aligned} \Pr[|C \cap \text{Corr}| \geq B] &= \Pr[|C \cap S| + |C \cap \text{Corr}^{\text{Before}}| \geq B] \\ &= \Pr[|C \cap S| + |C' \cap \text{Corr}^{\text{Before}}| \geq B] \\ &= \Pr[|(C \setminus \text{Corr}^{\text{Before}}) \cap S| + |C' \cap \text{Corr}^{\text{Before}}| \geq B] \\ &= \Pr[|(C' \setminus \text{Corr}^{\text{Before}}) \cap S| + |C' \cap \text{Corr}^{\text{Before}}| \geq B] \\ &= \Pr[|C' \cap \text{Corr}| \geq B]. \end{aligned}$$

Therefore, the probability that both hybrids output 1 is the same:

$$\Pr[\mathcal{H}_{\text{Rep-NIKE-}n} = 1] = \Pr[\mathcal{H}_{\text{Resample}} = 1].$$

Finally, let us analyze the probability that $\mathcal{H}_{\text{Resample}}$ outputs 1. Fix an arbitrary set Corr with $|\text{Corr}| \leq t$ and recall that $t < (1/2 - \epsilon)n$. Sampling C' can be seen as sampling elements from $[n]$ one by one without replacement for m draws. For $i \in [n]$, let Y_i be the indicator variable that is 1, if $i \in C'$ and 0 else. For $i \in [m]$, let Z_i be the indicator variable that is 1, if the i -th draw is in Corr and 0 else. Let Z be the sum of all Z_i , i.e., $Z = \sum_i Z_i$. Note that Z is the number of elements in the intersection $C' \cap \text{Corr}$ and therefore we can also write Z as the sum of the Y_i over all $i \in \text{Corr}$. Observe that

$$\Pr[Y_i = 1] = \frac{\binom{n-1}{m-1}}{\binom{n}{m}} = \frac{m}{n}.$$

In combination, we get that the expected number of elements in the intersection satisfies

$$\mathbf{E}[Z] = \mathbf{E}\left[\sum_{i \in \text{Corr}} Y_i\right] = \sum_{i \in \text{Corr}} \Pr[Y_i = 1] = |\text{Corr}| \cdot \frac{m}{n} \leq (1/2 - \epsilon)m =: \mu.$$

Next, we observe that

$$Z - \mathbf{E}[Z] \geq Z - \mu.$$

Therefore, if the hybrid outputs 1, i.e., $Z \geq m/2$, then

$$Z - \mathbf{E}[Z] \geq m/2 - (1/2 - \epsilon)m = m(1/2 - 1/2 + \epsilon) = m\epsilon.$$

Applying the Hoeffding bound for sampling without replacement [Hoe63], we thus get

$$\Pr[Z \geq m\epsilon] \leq \Pr[Z - \mathbf{E}[Z] \geq m\epsilon] \leq e^{-2(m\epsilon)^2/m} = e^{-2m\epsilon^2}.$$

From here we can conclude that

$$\text{Exp}_{\text{STE}, \mathcal{A}, B, t, id}^{\text{core}}(1^\lambda) \leq \text{negl}(\lambda) + \Pr[\mathcal{H}_{\text{Resample}} = 1] \leq \text{negl}(\lambda) + e^{-2m\epsilon^2},$$

which proves the statement. $\square$

Theorem 2 (Robustness). Assume that $H: \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ is modeled as a random oracle. Assume that SKE is correct (Definition 5), SS is correct (Definition 2), PKE is correct (Definition 8), NIKE is correct (Definition 11), and DS is correct (Definition 14). Assume that NIKE is one-pair malicious-key secure (Definition 12) and that PKE has indistinguishable keys (Definition 9). Assume that $\lg e \cdot 2m\epsilon^2 \geq \kappa$. Then, the (t, n) -silent threshold encryption scheme STE, as defined in Figure 4, is robust under one-shot adaptive corruptions for $t < (1/2 - \epsilon) \cdot n$, where $1/2 > \epsilon \geq 0$ is a constant.

Proof. We start with a short intuition and by recalling the security experiment $\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-AR}}(1^\lambda)$ from Figure 3. The adversary initially corrupts a set of parties and gets access to a decryption oracle and a corruption oracle. It also specifies a target message msg , which is then encrypted with respect to all encryption keys $\text{ek}_1, \dots, \text{ek}_n$. In particular, for the concrete scheme at hand, the ciphertext has the form

$$\text{ctxt} := (\widetilde{\text{ctxt}}, D, R, \text{pk}).$$

Here, $\widetilde{\text{ctxt}}$ is a symmetric key encryption of msg using a random key k , and D is a dictionary build via

$$E := \{(K_j, \text{ctxt}_j \| h_j \| w_j) \mid j \in C\}, \quad R \leftarrow \mathcal{R}, \quad D \leftarrow \text{Insert}(E, R)$$

where each ctxt_j encrypts a random value r_j with respect to the encryption key $\widetilde{\text{ek}}_j$, and $w_j = H(w_j) \oplus s_j$ for share s_j of k , and $C \subset [n]$ is a random committee of size m sampled during encryption, and each K_j is the shared NIKE key computed as $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$, where $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ is sampled during encryption. The tags, $h_j = H(r_j \| s_j)$ are essential for robustness. Namely, recall that the adversary wins the game if ctxt does not decrypt to msg , where honest parties provide their decryption shares honestly and the decryption shares of corrupted parties are provided by the adversary. Each decryption share is a triple (K, r, s) , and the decryption process ignores tuples for which (1) K is not in the dictionary D , or $H(r \| s) \neq \widehat{h}$ for $\widetilde{\text{ctxt}} \| \widehat{h} \| \widehat{w} \leftarrow \text{Get}(D, R, K)$. Assuming there are no collisions for H , this ensures that the reconstruction algorithm of the secret sharing only gets as input *valid* shares of k . Therefore, we only need to argue that there are enough such shares, and this can be done by arguing that there are enough honest parties left. We now make the details more formal. We start by defining three events in the game $\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-AR}}(1^\lambda)$:

- **ROColl:** This event occurs, if at any point in time there are random oracle queries $x \neq x'$ with $H(x) = H(x')$. This includes queries made by the security experiment itself.
- **SmallIntersect:** This event occurs, if at the end of the experiment, we have $|C \cap \text{Corr}| < m/2$.
- **Win:** This event occurs, if the experiment outputs 1. To recall, this happens if $|\text{Corr}| \leq t$ and $\text{Dec}(\text{ctxt}, \{\text{pdec}_i\}_{i \in [n]}) \neq \text{msg}$. Our goal is to upper bound the probability of this event.

We now proceed with three claims that constitute the proof.

Claim 3. We have $\Pr[\text{ROColl}] \leq \text{negl}(\lambda)$.

Proof of Claim. This follows from a standard birthday bound.

Claim 4. We have $\Pr[\neg \text{SmallIntersect}] \leq \text{negl}(\lambda) + \text{negl}(\kappa)$.

Proof of Claim. This claim follows from a simple reduction to the experiment in the core lemma (Lemma 1), and from the fact that we assume $\lg e \cdot 2m\epsilon^2 \geq \kappa$.

Claim 5. We have $\Pr[\text{Win} \mid \text{SmallIntersect} \wedge \neg \text{ROColl}] \leq \text{negl}(\kappa)$.

Proof of Claim. Consider the decryption process (algorithm Dec in Figure 4) and assume that $\neg \text{ROColl}$ and SmallIntersect occur. Denote by (r_j, s_j) the random values and shares of k that have been computed during encryption. By definition, the decryption process inputs the following set of shares into the reconstruction algorithm:

$$S := \left\{ s \mid \exists i: (K, r, s) = \text{pdec}_i \wedge H(r \| s) = \widehat{h} \text{ where } \widetilde{\text{ctxt}} \| \widehat{h} \| \widehat{w} \leftarrow \text{Get}(D, R, K) \right\}.$$

First, one can see that S is a subset of $\{s_j\}_{j \in C}$. This is because **ROColl** does not occur. Precisely, if K has not been inserted into D , then $\text{Get}(D, R, K)$ returns $\perp$ and s is for sure not in S . If K has been inserted

into D , then the encryption process ensured that $H(r_i \| s_i) = \hat{h}$, so because ROColl does not occur, $s = s_i$. Second, because SmallIntersect does not occur, PKE is correct, NIKE is correct, and DS is correct (except with probability $\text{negl}(\kappa)$) one can see that S has size at least $m/2$. Therefore, the decryption process recovers k by correctness of SS . Consequently, by correctness of SKE , the decryption process recovers msg , and the adversary cannot win the game. $\square$

Theorem 6 (Privacy). Assume that $H: \{0,1\}^* \rightarrow \{0,1\}^{2\lambda}$ is modeled as a random oracle. Assume that $\lg e \cdot 2m\epsilon^2 \geq \kappa$. Assume that SS is perfectly private (Definition 3). Assume that SKE is semantically secure (Definition 6). Assume that NIKE is one-pair malicious-key secure (Definition 12) and that PKE has indistinguishable keys (Definition 9). Then, the (t, n) -silent threshold encryption scheme STE , as defined in Figure 4, satisfies privacy under one-shot adaptive corruptions for $t < (1/2 - \epsilon) \cdot n$, where $1/2 > \epsilon \geq 0$ is a constant.

Proof. We prove the claim via a sequence of hybrid experiments.

$\mathcal{H}_{\text{orig}}$	$\text{Dec0}(\text{ctxt}, i)$
<pre> 1: $I \leftarrow \mathcal{A}(1^\lambda)$ 2: $\text{Corr} := I, \text{PartDecs} := \emptyset$ 3: for $i \in [n] \setminus \text{Corr}$: 4: $(\widetilde{\text{ek}}_i, \widetilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 5: $\text{ek}_i := (\widetilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\widetilde{\text{dk}}_i, \text{sk}_i)$ 6: $((\text{ek}_i = (\widetilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Corr0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 7: if $\exists i, j \in [n]$ s.t. $\text{id}_i = \text{id}_j \wedge i \neq j$: return 0 8: $b \leftarrow \{0, 1\}$ 9: $C \leftarrow \{S \subset [n] \mid S = m\}$ 10: $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 11: $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 12: $\widetilde{\text{ctxt}} \leftarrow \text{SKE.Enc}(k, \text{msg}_b)$ 13: $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 14: $E := \emptyset$ 15: for $j \in C$: 16: $r_j \leftarrow \{0, 1\}^\lambda$ 17: $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\widetilde{\text{ek}}_j, r_j)$ 18: $w_j := H(r_j) \oplus s_j, h_j := H(r_j \ s_j)$ 19: $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 20: $E := E \cup \{(K_j, \text{ctxt}_j \ h_j \ w_j)\}$ 21: $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 22: $\text{ctxt} := (\widetilde{\text{ctxt}}, D, R, \text{pk})$ 23: $S \leftarrow \mathcal{A}(\text{ctxt})$ 24: if $S \cap \text{Corr} \neq \emptyset$: return 0 25: $\text{Corr} := \text{Corr} \cup S$ 26: $b' \leftarrow \mathcal{A}(\{\text{dk}_i\}_{i \in S})$ 27: if $\text{Corr} \leq t \wedge b = b'$: return 1, else : return 0 </pre>	<pre> 1: if $i \in \text{Corr}$: return $\perp$ 2: else : 3: $\text{PartDecs} := \text{PartDecs} \cup \{\text{ctxt}\}$ 4: parse ctxt as $(\widetilde{\text{ctxt}}, D, R, \text{pk})$ 5: $K \leftarrow \text{NIKE.SharedKey}(\text{pk}, 0, \text{sk}_i, \text{id}_i)$ 6: $\widetilde{\text{ctxt}} \ \hat{h} \ \hat{w} \leftarrow \text{Get}(D, R, K)$ 7: if $\widetilde{\text{ctxt}} \ \hat{h} \ \hat{w} = \perp$: 8: return $\perp$ 9: $r \leftarrow \text{PKE.Dec}(\widetilde{\text{dk}}_i, \widetilde{\text{ctxt}})$ 10: $s := H(r) \oplus w$ 11: return (K, r, s) </pre>
	<pre> Cor0(i) 1: if $i \in \text{Corr}$: return $\perp$ 2: else : 3: $\text{Corr} := \text{Corr} \cup \{i\}$ 4: return dk_i </pre>

Fig. 9. Hybrid $\mathcal{H}_{\text{orig}}$ in the proof of Theorem 6.

$\mathcal{H}_{\text{orig}}$. This is the original privacy experiment $\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-CCA1}}$ for STE as defined in Figure 4. We present it formally in Figure 9. We recall this game briefly: After specifying an initial set of corruptions, the adversary gets honestly generated keys ek_i for each honest party i , where $(\text{ek}_i, \text{dk}_i) \leftarrow \text{Gen}(1^\lambda, \text{id}_i)$. In the scheme of interest, these keys have the form $\text{ek}_i = (\widetilde{\text{ek}}_i, \text{pk}_i)$ and $\text{dk}_i = (\widetilde{\text{dk}}_i, \text{sk}_i)$ for $(\widetilde{\text{ek}}_i, \widetilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda)$ and $(\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$. The adversary can pick the keys $\text{ek}_i = (\widetilde{\text{ek}}_i, \text{pk}_i)$

$\mathcal{H}_{\text{fewcorr}}$

```

1:  $I \leftarrow \mathcal{A}(1^\lambda)$ 
2:  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
3: for  $i \in [n] \setminus \text{Corr}$  :
4:    $(\tilde{\text{ek}}_i, \tilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 
5:    $\text{ek}_i := (\tilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\tilde{\text{dk}}_i, \text{sk}_i)$ 
6:    $((\text{ek}_i = (\tilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 
7:   if  $\exists i, j \in [n]$  s.t.  $\text{id}_i = \text{id}_j \wedge i \neq j$  : return 0
8:    $b \leftarrow \{0, 1\}$ 
9:    $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
10:   $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 
11:   $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
12:   $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg}_b)$ 
13:   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
14:   $E := \emptyset$ 
15:  for  $j \in C$  :
16:     $r_j \leftarrow \{0, 1\}^\lambda$ 
17:     $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_j, r_j)$ 
18:     $w_j := H(r_j) \oplus s_j, h_j := H(r_j \| s_j)$ 
19:     $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 
20:     $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
21:   $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
22:   $\text{ctxt} := (\text{ctxt}, D, R, \text{pk})$ 
23:   $S \leftarrow \mathcal{A}(\text{ctxt})$ 
24:  if  $S \cap \text{Corr} \neq \emptyset$  : return 0
25:   $\text{Corr} := \text{Corr} \cup S$ 
26:   $b' \leftarrow \mathcal{A}(\{\text{dk}_i\}_{i \in S})$ 
27:  if  $|C \cap \text{Corr}| \geq m/2$  : return 0
28:  if  $|\text{Corr}| \leq t \wedge b = b'$  : return 1, else : return 0

```

 $\mathcal{H}_{\text{noQry}}$

```

1:  $I \leftarrow \mathcal{A}(1^\lambda)$ 
2:  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
3: for  $i \in [n] \setminus \text{Corr}$  :
4:    $(\tilde{\text{ek}}_i, \tilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 
5:    $\text{ek}_i := (\tilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\tilde{\text{dk}}_i, \text{sk}_i)$ 
6:    $((\text{ek}_i = (\tilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 
7:   if  $\exists i, j \in [n]$  s.t.  $\text{id}_i = \text{id}_j \wedge i \neq j$  : return 0
8:    $b \leftarrow \{0, 1\}$ 
9:    $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
10:   $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 
11:   $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
12:   $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg}_b)$ 
13:   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
14:   $E := \emptyset$ 
15:  for  $j \in C$  :
16:     $r_j \leftarrow \{0, 1\}^\lambda$ 
17:     $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_j, r_j)$ 
18:    if  $j \notin \text{Corr}$  :  $w_j, h_j \leftarrow \{0, 1\}^{2\lambda}$ 
19:    else :  $w_j := H(r_j) \oplus s_j, h_j := H(r_j \| s_j)$ 
20:     $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 
21:     $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
22:   $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
23:   $\text{ctxt} := (\text{ctxt}, D, R, \text{pk})$ 
24:   $S \leftarrow \mathcal{A}(\text{ctxt})$ 
25:  if  $S \cap \text{Corr} \neq \emptyset$  : return 0
26:   $\text{Corr} := \text{Corr} \cup S$ 
27:  for  $j \in C \cap \text{Corr}$  :  $\text{Program } H(r_j \| s_j) := h_j, H(r_j) := w_j \oplus s_j$ 
28:   $b' \leftarrow \mathcal{A}(\{\text{dk}_i\}_{i \in S})$ 
29:  if  $|C \cap \text{Corr}| \geq m/2$  : return 0
30:  if  $|\text{Corr}| \leq t \wedge b = b'$  : return 1, else : return 0

```

Fig. 10. Hybrids $\mathcal{H}_{\text{fewcorr}}$ (left) and $\mathcal{H}_{\text{noQry}}$ (right) in the proof of [Theorem 6](#). Oracles Cor0 and Dec0 are as in [Figure 9](#).

$\mathcal{H}_{\text{shareZero}}$

```

1 :  $I \leftarrow \mathcal{A}(1^\lambda)$ 
2 :  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
3 : for  $i \in [n] \setminus \text{Corr}$  :
4 :    $(\tilde{\text{ek}}_i, \tilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 
5 :    $\text{ek}_i := (\tilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\tilde{\text{dk}}_i, \text{sk}_i)$ 
6 :    $((\text{ek}_i = (\tilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 
7 :   if  $\exists i, j \in [n]$  s.t.  $\text{id}_i = \text{id}_j \wedge i \neq j$  : return 0
8 :    $b \leftarrow \{0, 1\}$ 
9 :    $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
10 :   $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 
11 :   $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
12 :   $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg}_b)$ 
13 :   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(0), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
14 :   $E := \emptyset$ 
15 :  for  $j \in C$  :
16 :     $r_j \leftarrow \{0, 1\}^\lambda$ 
17 :     $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_j, r_j)$ 
18 :    if  $j \notin \text{Corr}$  :  $w_j, h_j \leftarrow \{0, 1\}^{2\lambda}$ 
19 :    else :  $w_j := H(r_j) \oplus s_j, h_j := H(r_j \| s_j)$ 
20 :     $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 
21 :     $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
22 :   $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
23 :   $\text{ctxt} := (\text{ctxt}, D, R, \text{pk})$ 
24 :   $S \leftarrow \mathcal{A}(\text{ctxt})$ 
25 :  if  $S \cap \text{Corr} \neq \emptyset$  : return 0
26 :   $\text{Corr} := \text{Corr} \cup S$ 
27 :  for  $j \in C \cap \text{Corr}$  : Program  $H(r_j \| s_j) := h_j, H(r_j) := w_j \oplus s_j$ 
28 :   $b' \leftarrow \mathcal{A}(\{\text{dk}_i\}_{i \in S})$ 
29 :  if  $|C \cap \text{Corr}| \geq m/2$  : return 0
30 :  if  $|\text{Corr}| \leq t \wedge b = b'$  : return 1, else : return 0

```

$\mathcal{H}_{\text{encZero}}$

```

1 :  $I \leftarrow \mathcal{A}(1^\lambda)$ 
2 :  $\text{Corr} := I, \text{PartDecs} := \emptyset$ 
3 : for  $i \in [n] \setminus \text{Corr}$  :
4 :    $(\tilde{\text{ek}}_i, \tilde{\text{dk}}_i) \leftarrow \text{PKE.Gen}(1^\lambda), (\text{pk}_i, \text{sk}_i) \leftarrow \text{NIKE.Gen}(1^\lambda, \text{id}_i)$ 
5 :    $\text{ek}_i := (\tilde{\text{ek}}_i, \text{pk}_i), \text{dk}_i := (\tilde{\text{dk}}_i, \text{sk}_i)$ 
6 :    $((\text{ek}_i = (\tilde{\text{ek}}_i, \text{pk}_i), \text{id}_i)_{i \in I}, \text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}^{\text{Cor0}, \text{Dec0}}((\text{ek}_i, \text{id}_i)_{i \in [n] \setminus \text{Corr}})$ 
7 :   if  $\exists i, j \in [n]$  s.t.  $\text{id}_i = \text{id}_j \wedge i \neq j$  : return 0
8 :    $b \leftarrow \{0, 1\}$ 
9 :    $C \leftarrow \{S \subset [n] \mid |S| = m\}$ 
10 :   $k \leftarrow \text{SKE.Gen}(1^\lambda)$ 
11 :   $(\text{pk}, \text{sk}) \leftarrow \text{NIKE.Gen}(1^\lambda)$ 
12 :   $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg}_0)$ 
13 :   $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(0), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$ 
14 :   $E := \emptyset$ 
15 :  for  $j \in C$  :
16 :     $r_j \leftarrow \{0, 1\}^\lambda$ 
17 :     $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_j, r_j)$ 
18 :    if  $j \notin \text{Corr}$  :  $w_j, h_j \leftarrow \{0, 1\}^{2\lambda}$ 
19 :    else :  $w_j := H(r_j) \oplus s_j, h_j := H(r_j \| s_j)$ 
20 :     $K_j \leftarrow \text{NIKE.SharedKey}(\text{pk}_j, \text{id}_j, \text{sk}, 0)$ 
21 :     $E := E \cup \{(K_j, \text{ctxt}_j \| h_j \| w_j)\}$ 
22 :   $R \leftarrow \mathcal{R}, D \leftarrow \text{Insert}(E, R)$ 
23 :   $\text{ctxt} := (\text{ctxt}, D, R, \text{pk})$ 
24 :   $S \leftarrow \mathcal{A}(\text{ctxt})$ 
25 :  if  $S \cap \text{Corr} \neq \emptyset$  : return 0
26 :   $\text{Corr} := \text{Corr} \cup S$ 
27 :  for  $j \in C \cap \text{Corr}$  : Program  $H(r_j \| s_j) := h_j, H(r_j) := w_j \oplus s_j$ 
28 :   $b' \leftarrow \mathcal{A}(\{\text{dk}_i\}_{i \in S})$ 
29 :  if  $|C \cap \text{Corr}| \geq m/2$  : return 0
30 :  if  $|\text{Corr}| \leq t \wedge b = b'$  : return 1, else : return 0

```

Fig. 11. Hybrids $\mathcal{H}_{\text{shareZero}}$ (left) and $\mathcal{H}_{\text{encZero}}$ (right) in the proof of [Theorem 6](#). Oracles Cor0 and Dec0 are as in [Figure 9](#).

(and identities) for every initially corrupted party i . It does so after seeing the honest keys and after interacting with a corruption oracle and a partial decryption oracle. In addition to these keys of initially corrupted parties, the adversary also outputs two messages $\text{msg}_0, \text{msg}_1$. The game ensures that all identities are distinct and samples a bit $b \in \{0, 1\}$ at random. It then encrypts msg_b as $\text{ctxt} \leftarrow \text{Enc}((\text{ek}_1, \text{id}_1), \dots, (\text{ek}_n, \text{id}_n), \text{msg}_b)$ and gives ctxt to the adversary. The adversary (without any further interaction with oracles) outputs a final set of corruptions and obtains their keys dk_i . The adversary then has to guess the bit b , and it wins if it made less than t corruptions and the guess is correct. Let us take a closer look at the encryption of the challenge ciphertext: here, the game samples a random committee $C \subset [n]$ of size m , a random symmetric key $k \leftarrow \text{SKE.Gen}(1^\lambda)$, and a fresh NIKE key pair $(pk, sk) \leftarrow \text{NIKE.Gen}(1^\lambda)$. It shares the key k and randomly permutes the shares: That is, it computes $(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k)$ and $(s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m)$. Then, for each party j in the committee, it samples a random string $r_j \leftarrow \{0, 1\}^\lambda$ and computes $\text{ctxt}_j \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_j, r_j)$, $w_j := H(r_j) \oplus s_j$, and $h_j := H(r_j \| s_j)$. It uses the NIKE to derive a shared key uses this key to insert $\text{ctxt}_j \| h_j \| w_j$ into a dictionary D which is part of the ciphertext. Both the dictionary and the NIKE can be largely ignored for the purpose of the proof, i.e., we can think of writing each $\text{ctxt}_j \| h_j \| w_j$ directly into the ciphertext. The NIKE is only essential in the proof of the core lemma (Lemma 1), which we will apply as a black box. The ciphertext also contains a symmetric encryption $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg}_b)$.

Intuitively, to remove information about b from the ciphertext, we will proceed in the following steps: (1) remove information about k from the triples $\text{ctxt}_j \| h_j \| w_j$ using the random oracle, security of PKE and security of the secret sharing scheme, and (2) apply security of SKE. Step (1) requires care due to adaptive corruptions, and requires the use of our core lemma. By definition, we have

$$\Pr \left[\text{Exp}_{\text{STE}, \mathcal{A}, t, \text{id}}^{\text{STE-CCA1}} = 1 \right] = \Pr [\mathcal{H}_{\text{orig}} = 1].$$

$\mathcal{H}_{\text{fewcorr}}$. This hybrid is the same as the hybrid before, but we change the winning condition. Namely, we let the hybrid output 0 if by the end of the experiment, we have $|C \cap \text{Corr}| \geq m/2$, i.e., the adversary manages to corrupt at least half of the parties in the committee. Otherwise, the hybrid determines its output as before. The formal description of the hybrid is given in Figure 10. To argue that the two hybrids are indistinguishable, we will use our core lemma (Lemma 1). Concretely, we present a reduction $\mathcal{B}$ that runs in the experiment defined in the code lemma, and simulates $\mathcal{H}_{\text{fewcorr}}$ resp. $\mathcal{H}_{\text{orig}}$ to the adversary. The reduction gets as input the security parameter and starts running the adversary. It outputs the same set of initial corruptions to the experiment as the adversary does. It forwards keys and identities of initially uncorrupted parties to the adversary, and uses its own partial decryption and corruption oracle to provide the respective oracles to the adversary. When the adversary outputs keys ek_i and identities id_i for every initially corrupted party i and two messages $\text{msg}_0, \text{msg}_1$, the reduction flips bit $b \leftarrow \{0, 1\}$ and outputs the ek_i, id_i , and message msg_b to the experiment from the core lemma. It gets back the challenge ciphertext and passes it to the adversary. Without any further interaction with any oracle, the adversary then outputs its set of one-shot corruptions S , which the reduction outputs to the core lemma experiment. The reduction then terminates. Clearly, the reduction is efficient and perfectly simulates the experiment to the adversary up to the point where it terminates. Note that the two hybrids $\mathcal{H}_{\text{orig}}$ and $\mathcal{H}_{\text{fewcorr}}$ only differ if $|C \cap \text{Corr}| \geq m/2$ by the end of the game. Whether this event occurs is fully determined at the point where the adversary outputs S and when the reduction terminates. It is therefore clear that

$$\begin{aligned} |\Pr [\mathcal{H}_{\text{orig}} = 1] - \Pr [\mathcal{H}_{\text{fewcorr}} = 1]| &\leq \Pr [\text{Exp}_{\text{STE}, \mathcal{B}, B, t, \text{id}}^{\text{core}}(1^\lambda) = 1] \\ &\leq \text{negl}(\lambda) + \text{negl}(\kappa), \end{aligned}$$

where $B = m/2$ and we have used the bound from the core lemma with $\lg e \cdot 2m\epsilon^2 \geq \kappa$.

$\mathcal{H}_{\text{noQry}}$. In this hybrid, we consider parties j that are not corrupted and in committee at the time the challenge ciphertext is generated: For those, we change how w_j and h_j are computed. Namely, we now sample them as $w_j, h_j \leftarrow \{0, 1\}^{2\lambda}$ and reprogram the random oracle consistently in case the party is corrupted later. Concretely, after the adversary returns the set S of one-shot corruptions, the experiment updates $\text{Corr} := \text{Corr} \cup S$ as before, and then for each $j \in C \cap \text{Corr}$, it programs $H(r_j \| s_j) := h_j$ and $H(r_j) := w_j \oplus s_j$. We note that this change only affects $j \in S \cap C$. The full hybrid is given in Figure 10. We will show indistinguishability in the following lemma.

Lemma 7 (Indistinguishability of $\mathcal{H}_{\text{fewcorr}}$ and $\mathcal{H}_{\text{noQry}}$). Assume that PKE has indistinguishable keys (Definition 9) and use the notation as in the proof of Theorem 6. Then, for any PPT adversary $\mathcal{A}$, it holds that

$$|\Pr[\mathcal{H}_{\text{fewcorr}} = 1] - \Pr[\mathcal{H}_{\text{noQry}} = 1]| \leq \text{negl}(\lambda).$$

We prove the lemma in Section A, and only give a proof sketch here. To argue indistinguishability, it is sufficient to bound the probability of the bad event that the adversary queries $H(r_j \| s_j)$ or $H(r_j)$ before j is corrupted. Intuitively, we want to use the security of PKE to argue that the adversary learns nothing about r_j and then rely on the entropy of r_j . However, there is a technical subtlety: if we switch the ciphertext ctxt_j that encrypts r_j to a ciphertext of a default plaintext, but the adversary corrupts party j afterwards, the change is clearly noticeable. To deal with this, we first consider a hybrid (specific to a fixed j) that immediately returns 1 if such a bad random oracle query occurs, and immediately outputs 0 if party j is corrupted. The probability that a bad query happens is at most the probability that this hybrid outputs 1. Then, we remove r_j from the ciphertext in *this hybrid*, in which corruption queries never need to be answered. We refer the reader to Section A for more details.

$\mathcal{H}_{\text{shareZero}}$. In this hybrid, we change how the shares are computed. Namely, while before we have computed them as

$$(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(k), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m),$$

we now compute them as

$$(\hat{s}_i)_{i=1}^m \leftarrow \text{Share}(0), (s_j)_{j \in C} \leftarrow \text{Shuffle}((\hat{s}_i)_{i=1}^m),$$

where we view 0 as being a default key. The full hybrid is presented in `creffig:privacy-proof:hybrid-sharezero`. We can argue indistinguishability using the (perfect) privacy of the secret sharing scheme SS (Definition 3). Note that if $|C \cap \text{Corr}| \geq m/2$, both hybrids output 0. We sketch a reduction that succeeds conditioned on $|C \cap \text{Corr}| < m/2$: it would get as input $m/2 - 1$ out of m many shares $\hat{s}_i$. It would then simulate the hybrid for the adversary. The crucial observation is that due to the previous hybrids, we have the following property: Throughout the hybrid, the reduction only needs shares s_j for parties that are in the corrupted part of the committee $C \cap \text{Corr}$ by the end of the game. For those, it can always provide a random unused share of the $\hat{s}_i$. As $|C \cap \text{Corr}| < m/2$, the shares that the reduction obtained suffice. The reduction outputs whatever the game outputs, and so we get

$$\Pr[\mathcal{H}_{\text{noQry}} = 1] = \Pr[\mathcal{H}_{\text{shareZero}} = 1].$$

$\mathcal{H}_{\text{encZero}}$. We change how the symmetric ciphertext $\widetilde{\text{ctxt}}$ is generated. So far, it has been generated as $\text{ctxt} \leftarrow \text{SKE.Enc}(k, \text{msg}_b)$, but now we generate it as $\widetilde{\text{ctxt}} \leftarrow \text{SKE.Enc}(k, \text{msg}_0)$. That is, we always encrypt msg_0 . We present the hybrid formally in Figure 11. Due to the previous changes, the key k is not used in any other place than that. In particular, we can build a straight-forward reduction to the security of SKE (Definition 6) to argue that this change is indistinguishable. We get

$$|\Pr[\mathcal{H}_{\text{shareZero}} = 1] - \Pr[\mathcal{H}_{\text{encZero}} = 1]| \leq \text{negl}(\lambda).$$

Finally, the probability that adversary guesses b in $\mathcal{H}_{\text{encZero}}$ is $1/2$, as all information about b is removed:

$$\Pr[\mathcal{H}_{\text{encZero}} = 1] = \frac{1}{2}.$$

4 Instantiations and Evaluation

To evaluate the efficiency of our construction, we consider two possible instantiations. The first one is based elliptic curve cryptography, which allows for smaller key and ciphertext sizes, but is not secure against quantum adversaries. The second is based on a combination of lattice and isogeny cryptography and thus plausibly provides security against quantum adversaries, but has larger key and ciphertext sizes. For both instantiations, we assume a computational security parameter λ of ≈ 128 and a statistical

Plaintext Size	Instantiation	Corruptions	Sizes		Post-Quantum
			ek	ctxt	
500 KB	ElGamal	$t \leq n/4$ $t \leq n/3$	66 Bytes	532 KB 572 KB	✗
	Kyber	$t \leq n/4$ $t \leq n/3$	1024 Bytes	692 KB 931 KB	✓
1 MB	ElGamal	$t \leq n/4$ $t \leq n/3$	66 Bytes	1.032 MB 1.072 MB	✗
	Kyber	$t \leq n/4$ $t \leq n/3$	1024 Bytes	1.192 MB 1.431 MB	✓

Table 2. Efficiency evaluation of our silent threshold encryption scheme. The post quantum column indicates whether the instantiation is plausibly secure against quantum adversaries or not.

security parameter $\kappa = 40$. We consider the settings $t \leq n/3$ and $t \leq n/4$, which translate into $\epsilon \geq 1/6$ and $\epsilon \geq 1/4$, respectively. Our security analysis (e.g., [Theorem 6](#)) required that $\lg(e) \cdot 2m\epsilon^2 \geq \kappa$ such that the statistical term in the bound from [Lemma 1](#) becomes negligible. Hence, we need to set $m \geq 500$ and $m \geq 222$ for $t \leq n/3$ and $t \leq n/4$, respectively.

We heuristically instantiate the random oracle using SHA256, which has 32 byte outputs. As our symmetric key encryption scheme we use AES in counter mode with 12 byte nonces. Thus a B byte plaintext produces a $B + 12$ byte ciphertext.

For our static dictionary, we use the construction⁸ of Groot Koerkamp [\[Gro25\]](#). For storing m many V byte values, the data structure occupies approximately $m \cdot (V + 1/3)$ bytes. Recall that each value stored in the dictionary is comprised of a public key encryption ciphertext, a hash value, and a one-time pad of a 2λ bit value.

Elliptic Curve Instantiation. To instantiate our required public key encryption scheme we use hashed ElGamal encryption as a key encapsulation mechanism, which satisfies the required key indistinguishability notion under the gap-Computational Diffie Hellman assumption [\[KL14\]](#), in combination with AES as described above. Specifically, we instantiate hashed ElGamal using the secp256k1 curve, which results in 33 byte encryption keys and $33 + 16 + (B + 12) = 61 + B$ byte ciphertexts for B byte plaintext messages. Using the same curve, we use the hashed Diffie-Hellman key exchange as discussed in several prior works [\[OP01; CKS08; FHK+13\]](#) for our non-interactive key exchange, we get a 33 byte public key. The full encryption key of a party is comprised of the public NIKE key and the public key encryption key, thus of in total of size $33 + 33 = 66$ bytes. Given the above instantiation, our silent threshold encryption scheme’s ciphertext is comprised of a 33 byte NIKE key and a dictionary storing m many V byte values, where

$$V = \underbrace{32}_{\text{hash}} + \underbrace{32}_{\text{one-time pad}} + \underbrace{(16 + 61)}_{\text{public key encryption}} = 141$$

and thus the full ciphertext is

$$\underbrace{B + 12}_{\text{Symmetric encryption}} + \underbrace{33}_{\text{NIKE key}} + \underbrace{m \cdot (141 + 1/3)}_{\text{dictionary}} + \underbrace{16}_{\text{dictionary seed}} < B + 61 + m \cdot 142$$

bytes long.

Post-Quantum Instantiation. To instantiate our required public key encryption scheme we use the Kyber512 key encapsulation mechanism, which was shown to satisfy key indistinguishability by Maram and Xagawa [\[MX23\]](#), in combination with AES as specified above. As our non-interactive key exchange, we use the GA-HDH construction of Duman et al. [\[DHK+22\]](#), which is proven secure under an isogeny assumption that is considered plausibly secure against quantum adversaries. The size of a public key is 224 bytes long. With this instantiation, we obtain 800 byte encryption keys and $B + 12 + 768$ ciphertexts encrypting B byte plaintext messages. Thus we get

$$V = \underbrace{32}_{\text{hash}} + \underbrace{32}_{\text{one-time pad}} + \underbrace{(16 + 780)}_{\text{public key encryption}} = 860$$

⁸ Strictly speaking, they construct a minimal perfect hash function, but using such a hash function to address the cells in a array directly gives us a static dictionary.

and the full ciphertext is

$$\underbrace{B + 12}_{\text{Symmetric encryption}} + \underbrace{224}_{\text{NIKE key}} + \underbrace{m \cdot (860 + 1/3)}_{\text{dictionary}} + \underbrace{16}_{\text{dictionary seed}} < B + 252 + m \cdot 861$$

bytes long.

Evaluation. We present our evaluation results in [Table 2](#), where we calculate the ciphertext size for different parameter settings discussed above, assuming the plaintext is either 500 KB or 1 MB large. As one can see, the overhead in terms of bandwidth is minimal, since ciphertexts are minimally larger than the plaintext data itself.

References

- [ASB+21] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. “Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients”. In: *Financial Cryptography and Data Security - 25th International Conference, FC 2021, Virtual Event, March 1-5, 2021, Revised Selected Papers, Part II*. Ed. by Nikita Borisov and Claudia Díaz. Vol. 12675. Lecture Notes in Computer Science. Springer, 2021, pp. 279–298 (cited on Page 3).
- [BBD+01] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. “Key-Privacy in Public-Key Encryption”. In: *ASIACRYPT 2001*. Ed. by Colin Boyd. Vol. 2248. LNCS. Springer, Berlin, Heidelberg, Dec. 2001, pp. 566–582. DOI: [10.1007/3-540-45682-1_33](https://doi.org/10.1007/3-540-45682-1_33) (cited on Page 5).
- [BBH06] Dan Boneh, Xavier Boyen, and Shai Halevi. “Chosen Ciphertext Secure Public Key Threshold Encryption Without Random Oracles”. In: *CT-RSA 2006*. Ed. by David Pointcheval. Vol. 3860. LNCS. Springer, Berlin, Heidelberg, Feb. 2006, pp. 226–243. DOI: [10.1007/11605805_15](https://doi.org/10.1007/11605805_15) (cited on Page 1).
- [BGI+01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. “On the (Im)possibility of Obfuscating Programs”. In: *CRYPTO 2001*. Ed. by Joe Kilian. Vol. 2139. LNCS. Springer, Berlin, Heidelberg, Aug. 2001, pp. 1–18. DOI: [10.1007/3-540-44647-8_1](https://doi.org/10.1007/3-540-44647-8_1) (cited on Page 1).
- [CGJ+99] Ran Canetti, Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. “Adaptive Security for Threshold Cryptosystems”. In: *CRYPTO’99*. Ed. by Michael J. Wiener. Vol. 1666. LNCS. Springer, Berlin, Heidelberg, Aug. 1999, pp. 98–115. DOI: [10.1007/3-540-48405-1_7](https://doi.org/10.1007/3-540-48405-1_7) (cited on Page 1).
- [CKS08] David Cash, Eike Kiltz, and Victor Shoup. “The Twin Diffie-Hellman Problem and Applications”. In: *EUROCRYPT 2008*. Ed. by Nigel P. Smart. Vol. 4965. LNCS. Springer, Berlin, Heidelberg, Apr. 2008, pp. 127–145. DOI: [10.1007/978-3-540-78967-3_8](https://doi.org/10.1007/978-3-540-78967-3_8) (cited on Pages 5, 23).
- [Cle86] Richard Cleve. “Limits on the Security of Coin Flips when Half the Processors Are Faulty (Extended Abstract)”. In: *18th ACM STOC*. ACM Press, May 1986, pp. 364–369. DOI: [10.1145/12130.12168](https://doi.org/10.1145/12130.12168) (cited on Page 2).
- [CS98] Ronald Cramer and Victor Shoup. “A Practical Public Key Cryptosystem Provably Secure Against Adaptive Chosen Ciphertext Attack”. In: *CRYPTO’98*. Ed. by Hugo Krawczyk. Vol. 1462. LNCS. Springer, Berlin, Heidelberg, Aug. 1998, pp. 13–25. DOI: [10.1007/BFb0055717](https://doi.org/10.1007/BFb0055717) (cited on Page 5).
- [Des88] Yvo Desmedt. “Society and Group Oriented Cryptography: A New Concept”. In: *CRYPTO’87*. Ed. by Carl Pomerance. Vol. 293. LNCS. Springer, Berlin, Heidelberg, Aug. 1988, pp. 120–127. DOI: [10.1007/3-540-48184-2_8](https://doi.org/10.1007/3-540-48184-2_8) (cited on Page 1).
- [DF90] Yvo Desmedt and Yair Frankel. “Threshold Cryptosystems”. In: *CRYPTO’89*. Ed. by Gilles Brassard. Vol. 435. LNCS. Springer, New York, Aug. 1990, pp. 307–315. DOI: [10.1007/0-387-34805-0_28](https://doi.org/10.1007/0-387-34805-0_28) (cited on Page 1).
- [DGK+20] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. “Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability”. In: *2020 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2020, pp. 910–927. DOI: [10.1109/SP40000.2020.00040](https://doi.org/10.1109/SP40000.2020.00040) (cited on Page 3).
- [DHK+22] Julien Duman, Dominik Hartmann, Eike Kiltz, Sabrina Kunzweiler, Jonas Lehmann, and Doreen Riepel. “Group Action Key Encapsulation and Non-Interactive Key Exchange in the QROM”. In: *ASIACRYPT 2022, Part II*. Ed. by Shweta Agrawal and Dongdai Lin. Vol. 13792. LNCS. Springer, Cham, Dec. 2022, pp. 36–66. DOI: [10.1007/978-3-031-22966-4_2](https://doi.org/10.1007/978-3-031-22966-4_2) (cited on Page 23).
- [FHK+13] Eduarda S. V. Freire, Dennis Hofheinz, Eike Kiltz, and Kenneth G. Paterson. “Non-Interactive Key Exchange”. In: *PKC 2013*. Ed. by Kaoru Kurosawa and Goichiro Hanaoka. Vol. 7778. LNCS. Springer, Berlin, Heidelberg, 2013, pp. 254–271. DOI: [10.1007/978-3-642-36362-7_17](https://doi.org/10.1007/978-3-642-36362-7_17) (cited on Pages 5, 23).

- [FKS84] Michael L Fredman, János Komlós, and Endre Szemerédi. “Storing a sparse table with 0 (1) worst case access time”. In: *Journal of the ACM (JACM)* 31.3 (1984), pp. 538–544 (cited on Page 6).
- [GJK+99] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *EUROCRYPT’99*. Ed. by Jacques Stern. Vol. 1592. LNCS. Springer, Berlin, Heidelberg, May 1999, pp. 295–310. DOI: [10.1007/3-540-48910-X_21](https://doi.org/10.1007/3-540-48910-X_21) (cited on Page 1).
- [GKP+24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. “Threshold Encryption with Silent Setup”. In: *CRYPTO 2024, Part VII*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14926. LNCS. Springer, Cham, Aug. 2024, pp. 352–386. DOI: [10.1007/978-3-031-68394-7_12](https://doi.org/10.1007/978-3-031-68394-7_12) (cited on Pages 1–3).
- [Gro25] Ragnar Groot Koerkamp. “PtrHash: Minimal Perfect Hashing at RAM Throughput”. In: *23rd International Symposium on Experimental Algorithms, SEA 2025, July 22-24, 2025, Venice, Italy*. Ed. by Petra Mutzel and Nicola Prezza. Vol. 338. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 21:1–21:21 (cited on Page 23).
- [Hoe63] Wassily Hoeffding. “Probability inequalities for sums of bounded random variables”. In: *Journal of the American statistical association* 58.301 (1963), pp. 13–30 (cited on Page 16).
- [HSW24] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. “Foundations of Data Availability Sampling”. In: *IACR Communications in Cryptography* (2024) (cited on Page 3).
- [JL00] Stanislaw Jarecki and Anna Lysyanskaya. “Adaptively Secure Threshold Cryptography: Introducing Concurrency, Removing Erasures”. In: *EUROCRYPT 2000*. Ed. by Bart Preneel. Vol. 1807. LNCS. Springer, Berlin, Heidelberg, May 2000, pp. 221–242. DOI: [10.1007/3-540-45539-6_16](https://doi.org/10.1007/3-540-45539-6_16) (cited on Page 1).
- [KL14] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Third. Chapman and Hall, CRC Press, 2014. ISBN: 978-0815354369 (cited on Pages 5, 23).
- [Lan23] Roman Langrehr. “On the Multi-user Security of LWE-Based NIKE”. In: *TCC 2023, Part IV*. Ed. by Guy N. Rothblum and Hoeteck Wee. Vol. 14372. LNCS. Springer, Cham, 2023, pp. 33–62. DOI: [10.1007/978-3-031-48624-1_2](https://doi.org/10.1007/978-3-031-48624-1_2) (cited on Page 5).
- [Mau05] Ueli M. Maurer. “Abstract Models of Computation in Cryptography (Invited Paper)”. In: *10th IMA International Conference on Cryptography and Coding*. Ed. by Nigel P. Smart. Vol. 3796. LNCS. Springer, Berlin, Heidelberg, Dec. 2005, pp. 1–12. DOI: [10.1007/11586821_1](https://doi.org/10.1007/11586821_1) (cited on Page 1).
- [MX23] Varun Maram and Keita Xagawa. “Post-quantum Anonymity of Kyber”. In: *PKC 2023, Part I*. Ed. by Alexandra Boldyreva and Vladimir Kolesnikov. Vol. 13940. LNCS. Springer, Cham, May 2023, pp. 3–35. DOI: [10.1007/978-3-031-31368-4_1](https://doi.org/10.1007/978-3-031-31368-4_1) (cited on Pages 5, 23).
- [NP24] Charlie Noyes and Guru Vamsi Policharla. *Removing the Relays*. <https://www.paradigm.xyz/2024/10/removing-the-relays>. Accessed: 2025-05-15. 2024 (cited on Page 2).
- [OP01] Tatsuoaki Okamoto and David Pointcheval. “The Gap-Problems: A New Class of Problems for the Security of Cryptographic Schemes”. In: *PKC 2001*. Ed. by Kwangjo Kim. Vol. 1992. LNCS. Springer, Berlin, Heidelberg, Feb. 2001, pp. 104–118. DOI: [10.1007/3-540-44586-2_8](https://doi.org/10.1007/3-540-44586-2_8) (cited on Page 23).
- [Ped91] Torben P. Pedersen. “A Threshold Cryptosystem without a Trusted Party (Extended Abstract) (Rump Session)”. In: *EUROCRYPT’91*. Ed. by Donald W. Davies. Vol. 547. LNCS. Springer, Berlin, Heidelberg, Apr. 1991, pp. 522–526. DOI: [10.1007/3-540-46416-6_47](https://doi.org/10.1007/3-540-46416-6_47) (cited on Page 1).
- [RSY21] Leonid Reyzin, Adam D. Smith, and Sophia Yakoubov. “Turning HATE into LOVE: Compact Homomorphic Ad Hoc Threshold Encryption for Scalable MPC”. In: *Cyber Security Cryptography and Machine Learning - 5th International Symposium, CSCML 2021, Be’er Sheva, Israel, July 8-9, 2021, Proceedings*. Ed. by Shlomi Dolev, Oded Margalit, Benny Pinkas, and Alexander A. Schwarzmann. Vol. 12716. Lecture Notes in Computer Science. Springer, 2021, pp. 361–378 (cited on Pages 1, 2).
- [SG98] Victor Shoup and Rosario Gennaro. “Securing Threshold Cryptosystems against Chosen Ciphertext Attack”. In: *EUROCRYPT’98*. Ed. by Kaisa Nyberg. Vol. 1403. LNCS. Springer, Berlin, Heidelberg, 1998, pp. 1–16. DOI: [10.1007/BFb0054113](https://doi.org/10.1007/BFb0054113) (cited on Page 1).
- [Sha79] Adi Shamir. “How to Share a Secret”. In: *Communications of the Association for Computing Machinery* 22.11 (Nov. 1979), pp. 612–613. DOI: [10.1145/359168.359176](https://doi.org/10.1145/359168.359176) (cited on Page 3).

- [Sho94] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: *35th FOCS*. IEEE Computer Society Press, Nov. 1994, pp. 124–134. DOI: [10.1109/SFCS.1994.365700](https://doi.org/10.1109/SFCS.1994.365700) (cited on Page 1).
- [Sho97] Victor Shoup. “Lower Bounds for Discrete Logarithms and Related Problems”. In: *EUROCRYPT’97*. Ed. by Walter Fumy. Vol. 1233. LNCS. Springer, Berlin, Heidelberg, May 1997, pp. 256–266. DOI: [10.1007/3-540-69053-0_18](https://doi.org/10.1007/3-540-69053-0_18) (cited on Page 1).

Appendix A Proof of Lemma 7

Proof (of Lemma 7). The reader may have another look at the two hybrids of interest, specified in Figure 10. We first recall the key difference between the two hybrids: both hybrids have the structure of the privacy game for silent threshold encryption scheme STE:

1. The adversary does some initial corruptions and can interact with a partial decryption and a corruption oracle.
2. The adversary specifies two messages msg_0 and msg_1 .
3. The game encrypts msg_b for a bit b .
4. The adversary can specify in one shot a final set of corruptions and then has to guess b . It wins if the guess is correct (and some other conditions hold).

The difference is in how the challenge ciphertext is generated: Namely, in the first hybrid, $\mathcal{H}_{\text{fewcorr}}$, the ciphertext is created honestly. That is, for each party j in the committee C , a random $r_j \leftarrow \{0, 1\}^\lambda$ is sampled and encrypted using PKE towards the encryption key ek_j , and then the share s_j for party j is encrypted and committed to as $w_j := H(r_j) \oplus s_j$ and $h_j := H(r_j \| s_j)$. In contrast, in hybrid $\mathcal{H}_{\text{noQry}}$, the values w_j and h_j are sampled at random if $j \notin \text{Corr}$ and the random oracle H is programmed consistently in case j is later corrupted in the one-shot corruptions. We now define a set of bad events:

- Event Bad_j for $j \in [n]$: this event occurs, if $j \in C$, $j \notin \text{Corr}$, and $H(r_j)$ or $H(r_j \| s_j)$ is queried by the adversary *before* j is ever corrupted. In particular, for $j \in C$, $j \notin \text{Corr}$, this includes the case where it is queried before the challenge ciphertext is created, the case where $j \in S$ and it is queried before the adversary outputs S , and the case where $j \notin S$ and it is queried at any point.

It is clear that the two hybrids only differ if such an event occurs. Thus, we get with a union bound

$$|\Pr[\mathcal{H}_{\text{fewcorr}} = 1] - \Pr[\mathcal{H}_{\text{noQry}} = 1]| \leq \sum_{j=1}^n \Pr[\text{Bad}_j].$$

In the following, we will bound the probability of each Bad_j . To this end, we fix a $j^* \in [n]$ for the rest of the proof. To bound the probability of Bad_{j^*} , we consider the following sequence of hybrids:

$\widehat{\mathcal{H}}_{j^*}$. This hybrid is exactly as $\mathcal{H}_{\text{fewcorr}}$, but with the following changes:

- If at any point in time, the event Bad_{j^*} occurs – this can be either when the challenge ciphertext is generated or when a random oracle query is made – then the experiment immediately terminates with output 1.
- If Bad_{j^*} can no longer occur, i.e., if j^* is not part of the committee, or it is corrupted or the experiment is completed without Bad_{j^*} occurring, the game immediately terminates with output 0.

One observation that we make about this hybrid is that it never needs to *answer* corruption queries for j^* : if Bad_{j^*} occurs before, then it has already terminated with output 1, and if it did not, then the experiment terminates immediately with output 0. In particular, the secret key for party j^* is only needed for partial decryption queries, which happen before the challenge ciphertext is created. By definition of the hybrid, we have

$$\Pr[\text{Bad}_j \text{ in } \mathcal{H}_{\text{fewcorr}}] = \Pr[\widehat{\mathcal{H}}_{j^*} = 1].$$

$\widehat{\mathcal{H}}'_{j^*}$. This is exactly as $\widehat{\mathcal{H}}_{j^*}$, except that we change the way ctxt_{j^*} is generated, if $j^* \in C$ and j^* is not corrupted at that point (otherwise the experiment would have already terminated). Before, it has been generated as $\text{ctxt}_{j^*} \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_{j^*}, r_{j^*})$, and now we generate it as $\text{ctxt}_{j^*} \leftarrow \text{PKE.Enc}(\tilde{\text{ek}}_{j^*}, 0^l)$, where 0^l is a default message of appropriate length. One can easily see that these two hybrids are indistinguishable using the assumption that PKE has indistinguishable keys (Definition 9). In particular, as explained above, a reduction will never need the decryption key for j^* . It can use its own decryption oracle to simulate partial decryptions. We get

$$\left| \Pr[\widehat{\mathcal{H}}_{j^*} = 1] - \Pr[\widehat{\mathcal{H}}'_{j^*} = 1] \right| \leq \text{negl}(\lambda).$$

Finally, we bound the probability that $\widehat{\mathcal{H}}'_{j^*}$ outputs 1. To this end, we note that in $\widehat{\mathcal{H}}'_{j^*}$, as long as the game did not terminate, the adversary obtains no information about r_{j^*} except via the random oracle outputs $H(r_{j^*})$ or $H(r_{j^*}||s_{j^*})$ which are used to compute the challenge ciphertext. As r_{j^*} is sampled uniformly at random from $\{0,1\}^\lambda$, the probability that any fixed random oracle query made by the adversary matches r_{j^*} or $r_{j^*}||s_{j^*}$ is $2^{-\lambda}$. In particular, with a union bound over all (polynomially many) random oracle queries, we get the desired

$$\Pr \left[\widehat{\mathcal{H}}'_{j^*} = 1 \right] \leq \text{negl}(\lambda).$$